

Real-Time Hand & Finger Tracker

with MediaPipe and OpenCV

A Complete Study Handbook



Python 3.9–3.12 • MediaPipe 0.10.14 • OpenCV 4.11.0.86 • NumPy < 2

Everything in this handbook is traced to real, runnable code:

`hand_logic.py`, `hand_tracker.py`, `tests/test_hand_logic.py`

How to Use This Handbook

This handbook assumes **almost no Python** and **no machine learning at all**. Every idea is introduced with a physical analogy first, then made exact with mathematics, then connected to the actual line of code that implements it. Nothing is hand-waved and nothing is left as an exercise.

Reading order

1. **Chapters 1–2** build the two backgrounds you are missing: the Python language, how images are stored in memory, what a neural network is, and how MediaPipe actually finds a hand. No geometry yet.
2. **Chapter 3** is the mathematical heart: the inverted y -axis, the thumb rules, the palm normal, and the two signal-processing filters.
3. **Chapter 4** is system design: why the project is split in two modules, how the per-frame cache works, and how the app survives a missing camera.
4. **Chapter 5** walks the source code block by block, quoting the real files.
5. **Appendices** hold the complete verbatim source, the verification checklist, and a glossary.

You do *not* need a webcam to read any of this, and you do not need a webcam to run most of the code: the project ships a headless `-image` mode that runs the entire pipeline on a still picture.

The four teaching cards

Throughout the handbook you will meet four coloured boxes. They always mean the same thing:

Python Concept 0.1 • Python language, from zero

Soft blue cards teach the Python language itself: what a variable is, what `self` means, why a deque is not a list. If you already know Python, you can skim these — but they also explain the *idiom*, not just the syntax.

ML Concept 0.1 • Computer vision and machine learning

Soft purple cards demystify the ML half: what a tensor is, what a convolutional layer does, what “inference” means. No mathematics of training is assumed or required.

Math Deep Dive 0.1 • Derivations step by step

Teal framed cards derive the geometry and signal processing used by the app, with every algebraic step shown and a worked numeric example. Nothing is asserted without proof.

Edge-Case Trap 0.1 • Physical failure modes

Amber cards with a red spine flag the ways a hand tracker breaks in the real world: palm inversion, the adducted “knife hand”, camera-package conflicts. These are the difference

between a demo and a tool.

How code is presented

Code in this handbook is **quoted directly from the project files**, not retyped. That means the line numbers printed in the margin are the real line numbers of the real file, so you can open `hand_logic.py`, jump to line 127, and be looking at exactly what is printed here. Chapter 5 quotes ranges; the appendices print each file whole.

A note on the printed colours. The listing panels use the project's own palette — midnight navy background, cyan keywords, amber strings. If you are reading a greyscale printout, keywords appear bold, strings appear in a lighter grey, and comments are italic.

Contents

How to Use This Handbook	2
1 Foundations for Absolute Beginners	5
1.1 Python from Scratch for Computer Vision	5
1.1.1 What is an interpreter, and how does Python run a file?	5
1.1.2 Variables and dynamic typing	5
1.1.3 Lists, tuples and dictionaries	6
1.1.4 Functions, arguments, return values and type hints	7
1.1.5 Object-oriented programming, simplified	8
1.1.6 Five special constructs used in this project	9
1.2 How Digital Images Work	11
1.2.1 Pixels, resolution and channels	11
1.2.2 Why OpenCV thinks in BGR	11
1.2.3 A frame is a three-dimensional array	12
1.3 Machine Learning Demystified	12
1.3.1 Rules plus data, versus data plus answers	12
1.3.2 What a neural network actually is	13
1.3.3 Training versus inference	13
1.3.4 Normalized coordinates, and the conversion to pixels	14
2 The Under-the-Hood ML Engine	15
2.1 The two-stage pipeline	15
2.2 What the model reports: the 21 landmarks	16
2.2.1 The landmark table	16
2.2.2 Why 21 points, and not a bounding box or a mask	17
2.3 The dependency matrix, and three package traps	17
2.3.1 Suppressing the deprecation noise	19
2.4 Confidence thresholds: what 0.7 and 0.6 actually control	19
2.4.1 static_image_mode and max_num_hands	20
2.5 Handedness, and why the label inverts	20
3 Mathematical and Geometric Algorithms	22
3.1 The screen coordinate system, and the inverted y-axis	22
3.1.1 Why a fingertip being “above” its knuckle means a smaller <i>y</i>	22
3.2 The thumb, and the search for invariance	24
3.2.1 The obvious rule, and exactly how it breaks	24
3.2.2 The distance rule	24
3.2.3 The knife hand: when the distance rule lies	26
3.2.4 Keeping the old rule reachable, and what happens with no label	27
3.3 The palm normal: which side of the hand am I seeing?	27

3.3.1	The sign convention, and how to remember it	28
3.4	Smoothing: the exponential moving average	29
3.5	Gestures and the debouncer	31
3.5.1	From five booleans to a name	31
3.5.2	Why a raw per-frame answer flickers	31
3.6	The complete decision path	32
4	System Design and Production Architecture	34
4.1	Architectural separation	34
4.1.1	Why the split exists	34
4.1.2	What the split buys, concretely	35
4.2	Single-pass telemetry caching	35
4.2.1	The problem: repeated work in a hot loop	35
4.2.2	The solution: one parse, one dictionary per hand	35
4.2.3	Invalid input, and the safe result	36
4.3	Camera fallback and hardware resiliency	37
4.4	Headless mode: verifying without a webcam	38
4.4.1	Continuous integration without a display	38
4.5	The alpha-blended HUD	39
4.6	Teardown and resource safety	40
4.7	The full data flow	40
5	Complete Walkthrough of the Source Code	42
5.1	hand_logic.py — the pure layer	42
5.1.1	Module docstring, imports and constants	42
5.1.2	The distance helper	43
5.1.3	Normalized to pixel conversion, and the bounding box	43
5.1.4	The palm normal	44
5.1.5	Reading the handedness label safely	45
5.1.6	Parsing a frame: the single-pass cache	45
5.1.7	The finger rules	46
5.1.8	Gesture classification	48
5.1.9	The debouncer	48
5.1.10	The exponential moving average	49
5.2	hand_tracker.py — the application	50
5.2.1	Module setup, the import guard, and constants	50
5.2.2	HandTracker: construction and the detection call	51
5.2.3	The accessor methods	52
5.2.4	The smoothed tip, orientation, and debounced gesture	53
5.2.5	The HUD: text assembly and drawing	54
5.2.6	The per-frame loop body	55
5.2.7	Screenshots, and the camera fallback chain	56
5.2.8	The live capture loop	57
5.2.9	Headless image mode	58
5.2.10	The command-line interface and entry point	59
5.3	tests/test_hand_logic.py — the offline test suite	61
5.3.1	The synthetic hands	61
5.3.2	The rotation-invariance test — the project’s key regression	62
5.3.3	The safe-result contract	62

5.3.4	The fake MediaPipe results	62
5.3.5	Parse tests	63
A	Complete Source Listings	66
A.1	hand_logic.py	66
A.2	hand_tracker.py	70
A.3	tests/test_hand_logic.py	77
B	Verification Checklist and Troubleshooting	84
B.1	Setup and first run	84
B.2	Live-mode acceptance tests	85
B.3	Troubleshooting	86
C	Glossary and Quick Reference	87
C.1	Glossary	87
C.2	Constants and defaults	88
C.3	Dependency reference	89
C.4	Files in the project	89

Foundations for Absolute Beginners

This chapter builds the two backgrounds you need before any hand tracking can make sense: the Python language (Section 1.1) and how pictures live inside a computer (Section 1.2). It closes with a plain-English account of what machine learning actually is (Section 1.3). Nothing here is specific to hands yet — that is deliberate. If you have never written a line of code, read every word. If you have written Python before, read the boxes anyway: they explain the *why* behind idioms you may already use by habit.

1.1 Python from Scratch for Computer Vision

1.1.1 What is an interpreter, and how does Python run a file?

When you type `python hand_tracker.py` and press Enter, you are not “starting a program” in the way you start a compiled `.exe`. You are starting a program called the *interpreter*, and handing it a text file.

Python Concept 1.1 • The interpreter, and why it matters here

An **interpreter** is a program that reads your text file and performs the instructions one statement at a time, top to bottom, translating as it goes.

A useful analogy: a **compiled** language is like translating an entire novel into another language and then publishing it — slow to prepare, fast to read. An **interpreted** language is like a simultaneous translator standing next to you — ready instantly, but working sentence by sentence.

Two practical consequences you will meet within the first five minutes:

- **Errors appear only when that line is reached.** If line 300 has a typo but your camera never opens, the program may die at line 40 and you will never see the line-300 problem. An interpreter does not read the whole file first.
- **The same file can behave differently on different runs** if it depends on something outside the file (a camera, a network, a random number). This is why the tracker logs “Camera 0 opened at 640x480” — so you know what the run actually did.

Python files are named with the `.py` suffix, and `python` is the interpreter. In this project there are two of them: `hand_logic.py` and `hand_tracker.py`.

1.1.2 Variables and dynamic typing

A variable is a name attached to a value. That is the whole idea.

```
1 fps = 28           # an int  (whole number)
2 alpha = 0.65       # a float (decimal number)
3 label = "Right"     # a str  (text)
4 is_up = True        # a bool  (True / False)
```

These four lines use the four basic types you will meet constantly in this project. Python calls its basic types *built-in types*, and the project uses exactly these:

Type	Example	Used in the project for
int	320	pixel coordinates, frame counters, finger counts
float	0.65	normalized coordinates, the palm normal Z , alpha values
str	"Right"	handedness labels, gesture names, file paths
bool	True	"is this finger extended?" — five per hand, every frame

Python Concept 1.2 • Dynamic typing: the name has a type, not the box

In C or Java you would write `int fps = 28;` — the *container* is declared as an integer and can never hold anything else.

Python is **dynamically typed**: the *name* simply points at a value, and the value knows its own type. This line is legal:

```
1 value = 5          # value points at an int
2 value = "hello"    # now the same name points at a string
```

The price of this freedom is that type mistakes are discovered at *run time*, not compile time. The project pays that price back with **type hints** and **tests** — see Section 1.1.4.

1.1.3 Lists, tuples and dictionaries

Nearly every bug in a hand tracker is really a data-shape bug: the wrong number of points, points in the wrong order, or a name that does not exist. So it is worth being precise about the three containers this project uses.

Lists — ordered, changeable

A **list** is an ordered sequence you can modify. Square brackets, comma separated.

```
1 FINGER_PAIRS = [(8, 6), (12, 10), (16, 14), (20, 18)]
2 states = [False] * 5          # five False values
3 states[0] = True              # now [True, False, False, False, False]
4 print(len(states))            # 5
```

Line 1 is taken from `hand_logic.py`: it is the list of (fingertip, knuckle) landmark index pairs for the four long fingers. Line 2 is the “safe result” the project falls back to whenever landmarks are missing — note that multiplying a list by an integer *repeats* it, it does not do arithmetic.

The most important property of a list here is **indexing**: `points[0]` is the first element, `points[8]` is the ninth. In this project, landmark indices are the entire language we speak — index 0 is the wrist, index 8 is the tip of the index finger, index 17 is the base of the pinky. If you remember one thing from this section, remember that *the index number is a name*.

Tuples — ordered, frozen

A **tuple** is like a list that cannot be changed. Round brackets.

```
1 tip = (320, 240)              # a single point: x, y
2 x, y = tip                    # unpacking: x becomes 320, y becomes 240
3 pixel = points[8]             # a tuple from the landmark list
4 dist = math.hypot(pixel[0] - other[0], pixel[1] - other[1])
```

A 2D point is naturally a tuple: it is a fixed pair, and it should not be accidentally mutated. The project uses tuples for coordinates and lists for collections of coordinates, so the shape of the data tells you what it is.

Dictionaries — named, lookup by key

A **dictionary** maps keys to values. Curly braces.

```
1 hand = {
2     "label": "Right",
3     "px": [(320, 420), (365, 400)],    # pixel landmarks
4     "palm_z": -8775.0,
5     "orientation": "Palm",
6 }
7
8 print(hand["label"])    # Right
9 print(hand["bbox"])    # KeyError! this key does not exist
```

This is precisely the structure `parse_hand_data` builds once per frame for each detected hand. A dictionary is the right choice because a hand has *named* properties whose order does not matter — and because a reader of the code can ask for `hand["orientation"]` instead of remembering “the fifth element”.

Edge-Case Trap 1.1 • `AttributeError` and `KeyError` — the two errors of this section

- `hand["bbox"]` raises `KeyError`: the name is not in the dictionary.
- `hand.label` raises `AttributeError`: dictionaries are indexed with `[]`, not with a dot. The dot syntax is for *objects* (Section 1.1.5).

These two messages account for a large share of real-world Python errors in computer-vision code, because MediaPipe results are objects while our cached hand data is a dictionary — the access syntax differs between them.

1.1.4 Functions, arguments, return values and type hints

A **function** is a named block of work with inputs and (usually) one output.

```
1 def euclidean_distance(a, b):
2     return math.hypot(a[0] - b[0], a[1] - b[1])
3
4 gap = euclidean_distance((0, 0), (3, 4))    # gap is 5.0
```

Reading that from left to right:

- `def` announces a function definition.
- `euclidean\_distance` is its name.
- `(a, b)` are its **parameters** — placeholders for the values it will receive.
- The **indented block** is the **body**.
- `return` hands a value back to whoever called it. A function without a `return` hands back a special value called `None`, meaning “nothing”.

Default arguments

```

1 def fingers_state(pixels, hand_label="Unknown", thumb_mode="distance"):
2     ...
3
4 fingers_state(points)           # label is "Unknown", mode "distance"
5 fingers_state(points, "Right")  # label is "Right"
6 fingers_state(points, thumb_mode="x")  # keyword argument: order does not matter

```

An argument with an = in the definition is **optional**. This is why the project’s public methods can be called simply — `tracker.find_positions(frame, 0)` — while still offering knobs such as `alpha=0.65`.

Type hints are documentation that can be checked

```

1 def to_pixel_landmarks(landmarks, width, height):
2     """Convert normalized landmarks to integer pixel coordinates."""
3     return [(int(lm.x * width), int(lm.y * height)) for lm in landmarks]

```

Python does not enforce the types written in a hint — it will happily run and crash later if you pass nonsense. The hint is a promise to the reader, and modern editors check those promises for you as you type, before you ever run the program. Throughout this handbook you will see hints such as `list[bool]` (a list of booleans), `tuple[int, int]` (a pair of integers — one pixel coordinate) and `list[dict]` (one dictionary per detected hand).

Reading a signature aloud is a genuinely useful skill. Take `fingers_state(pixel_landmarks, hand_label="Unknown", palm_z=None, thumb_mode="distance")`. Read as: “give me the pixel landmarks; optionally tell me which hand this is; give me five booleans back — one per finger, thumb first.” Once you can read signatures, the architecture of the whole project becomes legible.

1.1.5 Object-oriented programming, simplified

Everything so far has been *data* (values) and *procedures* (functions) kept apart. Object orientation glues them together.

Python Concept 1.3 • Class, object, self — the three words that block beginners

A **class** is a blueprint. An **object** (or *instance*) is a thing built from that blueprint. `self` is how the object refers to itself.

Analogy: a class is the architectural drawing of a house; an object is an actual house. You can build many houses from one drawing, and each has its own front door colour — its own data — while sharing the same structure.

```

1 class HandTracker:
2     def __init__(self, max_hands=2):
3         self.max_hands = max_hands  # store on THIS object
4         self.hands_data = []        # each object gets its own empty list
5
6     def hand_count(self):
7         return len(self.hands_data)
8
9 tracker = HandTracker()              # build an object from the blueprint
10 print(tracker.hand_count())         # 0

```

Line by line:

- `class HandTracker:` defines the blueprint. By convention, class names are capitalised.
- `__init__` is the **constructor**: it runs automatically when the object is built. The name has two underscores on each side — in Python that signals “special method that the language itself calls”.
- `self` is the first parameter of every method. Python passes the object in automatically, so `tracker.hand_count()` is really `HandTracker.hand_count(tracker)`. You never pass `self` yourself; you only declare it.
- `self.hands_data = []` creates a piece of state *belonging to this object*. Crucially, each object gets its own list. Two trackers would not share one hand cache.

Why does this project need a class at all? Because detection has **state**: the MediaPipe model object itself, the smoothing history, the gesture debouncers. A bare function would have to be handed all of that on every call. A class lets the state live next to the methods that use it — which is why the detector is `HandTracker`, and `find_hands`, `find_positions` and `fingers_up` are its methods.

1.1.6 Five special constructs used in this project

1. deque with a maximum length

```
1 from collections import deque
2 from hand_logic import GestureDebouncer
3
4 history = deque(maxlen=5)
5 history.append("Fist")           # ['Fist']
6 history.append("Peace")         # ['Fist', 'Peace']
7 # ... after five appends ...
8 history.append("Fist")          # the OLDEST entry is silently evicted
```

A deque (“deck”, double-ended queue) is a list optimised for adding and removing at both ends. The `maxlen` argument makes it a **sliding window**: it never grows beyond five entries, and it never needs to be trimmed by hand. That single feature is the entire implementation of the gesture debouncer in `hand_logic.py`: keep the last five gestures, count how many agree.

Python Concept 1.4 • Why not a list?

You could write `history.append(g); if len(history) > 5: history.pop(0)` — but `pop(0)` on a list shifts every remaining element, which is $O(n)$. A deque is designed for that operation and evicts in $O(1)$. It is also harder to forget. In a 30-frames-per-second loop, forgetting to trim a list is a slow memory leak that only shows up during a long demo.

2. List comprehensions

```
1 # the long way
2 pixels = []
3 for lm in landmarks:
4     pixels.append((int(lm.x * width), int(lm.y * height)))
5
6 # the comprehension
```

```
7 pixels = [(int(lm.x * width), int(lm.y * height)) for lm in landmarks]
```

Both blocks do the same thing. The second is the idiom you will see in this codebase wherever one collection is transformed into another. Read a comprehension as: “build this expression, for each item in this collection.”

```
3. if __name__ == "__main__":
```

```
1 def main():
2     ...
3
4 if __name__ == "__main__":
5     raise SystemExit(main())
```

Python Concept 1.5 • The most-asked Python question, answered properly

When Python runs a file *directly* (python hand_tracker.py), it sets a built-in variable called `__name__` to the string `"__main__"` inside that file.

When the same file is *imported* by another file (import hand_tracker), Python sets `__name__` to the module’s own name, `"hand_tracker"`.

So if `__name__ == "__main__"`: means **“run this block only if I am the program being started, not if I am being used as a library.”**

This project needs exactly that distinction for three reasons:

- The test suite imports hand_tracker to test its helper functions. If the capture loop ran at import time, pytest would try to open your webcam.
- `raise SystemExit(main())` converts the integer that `main()` returns into the process exit code, so CI can tell success (0) from failure (1).
- It keeps the top of the file importable for tooling without side effects.

4. Exceptions and finally

```
1 try:
2     while True:
3         ok, frame = capture.read()
4         ...
5 finally:
6     capture.release()          # ALWAYS runs: normal exit, error, or Ctrl+C
7     cv2.destroyAllWindows()
```

An **exception** is Python’s way of saying “I cannot continue”. `try` marks a region where you expect one; `finally` marks cleanup that must happen no matter how the region ends. The tracker’s capture loop uses `finally` to release the camera and close the window. Without it, an unplugged webcam or a Ctrl+C would leave the camera device locked, and the next run would fail with “could not open a webcam” until you rebooted — a classic and infuriating bug.

5. The walrus-free guard clause

```
1 def find_positions(self, frame, hand_index=0):
2     if 0 <= hand_index < len(self.hands_data):
3         return list(self.hands_data[hand_index]["px"])
4     return []
```

Note the *chained comparison* `0 <= i < len(...)`. Python evaluates it as `(0 <= i)` and `(i < len(...))`, which is why a negative index — which would otherwise mean “count from the end” — is rejected. This two-line pattern (“check, return early; otherwise fall through to the safe default”) is how the project removes an entire class of crashes: “`IndexError: list index out of range`” is impossible here.

1.2 How Digital Images Work

1.2.1 Pixels, resolution and channels

An image is a grid of coloured dots called **pixels** (short for “picture elements”). The grid has a width and a height, measured in pixels; together they are the **resolution**.

ML Concept 1.1 • From photons to numbers

A camera sensor measures, for each of its tiny light-sensitive cells, how much light arrived. The webcam in this project is asked for 640×480 cells (that is 307,200 pixels). For each cell we get three intensities: how much **red**, how much **green**, how much **blue** light arrived. Each intensity is stored as one byte — an integer from 0 (none) to 255 (full). Three bytes per pixel, 307,200 pixels: about 0.9 megabytes per frame, and at 30 frames per second roughly 27 MB/s flowing through your program. This is exactly why the project caches work per frame instead of recomputing it (Chapter 4). A colour is therefore an ordered triple. (255, 0, 0) is pure red in RGB order; (0, 0, 0) is black; (255, 255, 255) is white.

1.2.2 Why OpenCV thinks in BGR

Here is a fact that costs beginners hours: OpenCV — the library that reads your webcam — stores colour channels in the order **blue, green, red**, not red, green, blue. It is a historical inheritance from the 1990s BGR camera hardware OpenCV was written against.

MediaPipe, on the other hand, was trained on ordinary RGB images.

Edge-Case Trap 1.2 • The BGR/RGB mismatch: silently wrong, not loudly broken

If you feed a BGR image to an RGB model, nothing crashes. Every colour is simply permuted: red and blue swap. Skin becomes a strange blue-ish tint, and the detector’s accuracy quietly degrades — or drops to zero — on some skin tones and lighting conditions. It is the worst kind of bug: no exception, no error message, just worse results. This is why the pipeline contains exactly one explicit conversion, at exactly one place:

```
1 rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
2 results = self.hands.process(rgb)
```

Everything *before* that line is BGR (because it came from the camera and will be shown by `cv2.imshow`); everything *after* is MediaPipe’s business. Screen output stays in BGR, because `cv2.imshow` expects BGR. The conversion is a one-way door for the model only.

1.2.3 A frame is a three-dimensional array

This is the single most useful mental model in the whole project.

ML Concept 1.2 • The flipbook and the cube

Think of video as a **flipbook**: each page is one still image, and flipping fast enough produces motion. A single frame is one page.

Now zoom into one page. It is a grid of pixels. Add the three colour values per pixel and the page becomes a **three-dimensional block of numbers**:

- **axis 0** — the rows (height): 480 of them
- **axis 1** — the columns (width): 640 of them
- **axis 2** — the channels: 3 of them

In NumPy — Python’s numerical array library — a frame is described by its shape, and you can inspect it at any moment:

```
>>> frame.shape
(480, 640, 3)
>>> frame[0, 0]          # top-left pixel, all three channels
array([ 42,  42,  45], dtype=uint8)
>>> frame[240, 320]      # the pixel at the centre of the frame
array([118, 147, 201], dtype=uint8)
>>> frame[240, 320, 0]   # just the BLUE value of that pixel
118
```

Note the order: `frame[y, x]`, *row first, column second*. This is a genuine trap, and it is the reason the project writes `height, width = frame.shape[:2]` rather than the other way around. MediaPipe works in the opposite convention — it reports normalized (x, y) — so every conversion in this codebase is a place where the two conventions meet.

One sentence to memorise: a frame is a (height, width, 3) block of bytes, indexed [row, column, channel], and a landmark is a normalised (x, y) pair indexed the other way round. Most “my dot is in the wrong place” bugs are one of these two facts being forgotten.

1.3 Machine Learning Demystified

1.3.1 Rules plus data, versus data plus answers

ML Concept 1.3 • The inversion that defines machine learning

Traditional programming. You write the rules, you feed in data, you get answers out.

rules + data $\rightarrow$ answers

Example: “if the pixel’s blue value is greater than its red value, call it skin.” You wrote the rule. The computer applies it.

Machine learning. You supply data and the correct answers for that data. The computer *derives* the rules.

data + answers $\rightarrow$ rules

Example: you show the computer 30,000 photographs of hands together with the exact pixel location of every knuckle, and it works out its own rules for finding knuckles.

You cannot write the rule for “where is the pinky knuckle” by hand. There is no threshold on red, green or blue that finds it. That is precisely why machine learning is used for this half of the project — and why the other half (deciding whether a finger is up) is ordinary geometry that *can* be written by hand, and therefore *should* be.

1.3.2 What a neural network actually is

ML Concept 1.4 • Weights, layers, and feature detectors

Strip away the vocabulary and a neural network is a very long chain of multiply-and-add operations, with a non-linear squashing function between the steps. That is all.

- A **weight** is one number that gets multiplied by an input. A small network has thousands; a large one has billions.
- A **layer** is a group of weights applied together, producing a new set of numbers from the previous set.
- A **feature** is a pattern that a layer reacts to. Early layers react to edges and colour gradients. Middle layers react to combinations such as “a curved edge above a straight one”. Late layers react to whole structures: “this cluster of edges is a knuckle”.

An analogy that holds up better than most: think of a factory production line. Raw pixels go in at one end. At each station, workers check for one specific simple thing and pass on a summary. By the end of the line, the summary is abstract enough to answer the question — “the pinky knuckle is at (0.31, 0.62)”.

The important consequence for you: **each layer sees only the previous layer’s summary**, never the raw image. That is what makes the whole thing fast enough to run on a CPU at 30 FPS.

1.3.3 Training versus inference

ML Concept 1.5 • We only ever run the trained model

Training is the process of searching for the weights. It requires a large labelled dataset, a great deal of computation — usually GPUs for hours or days — and the ability to measure error and correct it. Training is what Google did once, in order to ship MediaPipe.

Inference is using a network whose weights are already fixed. Given an input, produce an output. No learning happens. It is a one-directional calculation.

This project is **100% inference**. The weights arrive inside the MediaPipe wheel as a .tflite file — a few megabytes of numbers — and the library loads them into memory when you write:

```
1 self.hands = mp_hands.Hands(
2     static_image_mode=mode,
3     max_num_hands=max_hands,
4     min_detection_confidence=det_conf,
5     min_tracking_confidence=track_conf,
6 )
```

Those four keyword arguments are *not* training settings. They choose how confident the model must be before it reports something. Chapter 2 covers what each one does and why the defaults are 0.7 and 0.6.

1.3.4 Normalized coordinates, and the conversion to pixels

This is the last prerequisite, and it is where the first real mathematics of the project appears.

MediaPipe does not report pixel positions. It reports **normalized coordinates**: numbers in the range $[0.0, 1.0]$, where 0.0 is the left (or top) edge of the image and 1.0 is the right (or bottom) edge.

Math Deep Dive 1.1 • From normalized coordinates to pixels

Why normalized? Because the model must not care about resolution. Whether the frame is 640×480 or 1920×1080 , a hand in the middle of the picture is at $x_{\text{norm}} = 0.5$. Normalizing makes one trained model valid on every camera, forever. The model predicts *relative* position; the exact pixel is an application concern.

The conversion. If a landmark sits at normalized position x_{norm} on an image W pixels wide, its pixel column is

$$x_{\text{px}} = x_{\text{norm}} \cdot W$$

and, since the image is H pixels tall,

$$y_{\text{px}} = y_{\text{norm}} \cdot H$$

Worked example. MediaPipe reports the index fingertip at $(x_{\text{norm}}, y_{\text{norm}}) = (0.5469, 0.4896)$ on the default 640×480 frame:

$$x_{\text{px}} = 0.5469 \times 640 = 350.0$$

$$y_{\text{px}} = 0.4896 \times 480 = 235.0$$

so the fingertip is drawn at pixel $(350, 235)$ — measured from the *top-left* corner.

Why the code truncates instead of rounding. The project writes

```
return [(int(lm.x * width), int(lm.y * height)) for lm in landmarks]
```

and `int()` *truncates* ($350.9 \rightarrow 350$) rather than rounds ($350.9 \rightarrow 351$). For drawing a filled circle of radius 10 pixels, a half-pixel bias is invisible, and truncation is measurably faster. What matters is that the value is an integer at all: OpenCV's drawing functions accept floats, but integer pixel coordinates make the cache smaller and the arithmetic exact when the geometry chapter computes distances and cross products.

Chapter summary. Python is an interpreted language whose lists, tuples and dictionaries give the project its data shapes. An image is a $(\text{height}, \text{width}, 3)$ block of bytes in BGR order, and MediaPipe speaks normalized RGB. Machine learning inverts the usual programming equation — data plus answers produces rules — and this project uses a model that has already been trained, only ever running inference. Landmarks arrive normalized and must be multiplied by the frame size to become pixels.

The Under-the-Hood ML Engine

Chapter 1 established that this project only ever runs *inference*: the weights were trained once, by someone else, and shipped inside the MediaPipe wheel. This chapter opens the box. It explains the two-stage architecture that makes real-time hand tracking possible on a CPU, what the model actually outputs, and the three dependency pins that will break your installation if you ignore them.

2.1 The two-stage pipeline

A naive design would run one enormous neural network over the entire 640×480 frame, every frame, looking for 21 landmarks. That design would be far too slow: most of the image is background, and re-searching the whole frame 30 times a second wastes almost all of the computation.

MediaPipe's solution is to split the problem into a cheap “where” and an expensive “what”.

ML Concept 2.1 • Stage 1 — BlazePalm: find the hand, coarsely

BlazePalm is a *single-shot detector*. Its job is not to find fingers; it is to answer one question, roughly: *is there a palm here, and where?*

It works by sliding a set of predefined boxes — called **anchors** — over a downsampled version of the frame, and for each anchor predicting two things:

1. a **score**: how likely is it that this box contains a palm?
2. an **offset**: how should the box be nudged and resized to fit better?

It predicts a palm rather than a whole hand on purpose. A palm is rigid: its shape and size vary little between people and poses, and — critically — it looks similar whether the hand is open, closed, or rotated. A fist and an open hand have nearly the same palm box. That robustness is what makes the detector reliable enough to hand off to stage two.

The output of stage 1 is one or more rectangles (plus a rotation angle, for hands tilted in the image plane) that are *likely* to contain a palm.

ML Concept 2.2 • Stage 2 — the landmark model: find the 21 points, precisely

Stage 2 takes the box from stage 1, **crops the frame to that box**, resizes the crop to the network's fixed input size, and runs a second, more expensive network whose only job is regression on 21 specific points.

Because the crop is small and tightly centred on the hand, the network sees the fingers at high effective resolution — a fingertip may occupy 30 pixels in the crop where it occupied 5 in the full frame. That is how it can predict a knuckle to a fraction of a pixel.

The output is 21 landmarks, each with three coordinates (x, y, z) normalised to $[0, 1]$ relative to the *full* frame — the library maps the crop coordinates back for you. The z value is relative depth, roughly “how far in front of or behind the wrist plane”, and it is what makes this a 3D system: fingers pointing at the camera still separate correctly in depth.

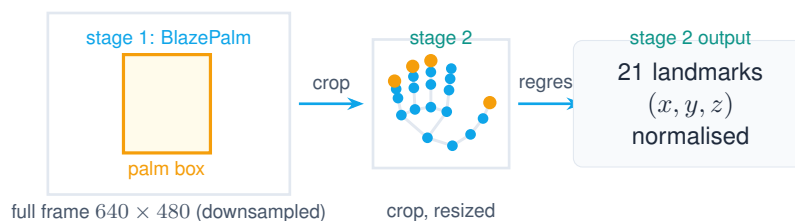


Figure 2.1. The two-stage detection pipeline. Stage 1 searches the whole frame for a palm box — cheap, because the image is downsampled and the target is rigid. Stage 2 runs a heavier network only inside that box. The split is what buys 30 FPS on a CPU.

The economic argument for two stages. Suppose the full frame costs 100 units of compute and a crop costs 10. Running the expensive network full-frame costs 100 per frame. Running a cheap detector once and the expensive network on a crop costs roughly $5 + 10 = 15$ — and when the hand is moving smoothly, MediaPipe can even skip the detector entirely for several frames and simply *track* the box, because it also predicts the hand’s position in the next frame. That is the difference between 8 FPS and 30 FPS, and it is why the design works at all.

2.2 What the model reports: the 21 landmarks

MediaPipe numbers the 21 landmarks in a fixed order. **These indices are the vocabulary of the entire project** — every rule in Chapter 3 is expressed as arithmetic on them, so they are worth learning properly.



Figure 2.2. The MediaPipe hand topology. The bold labels are the landmarks this project reasons about directly: the wrist anchors the palm normal, the thumb tip and IP joint drive the thumb rule, and indices 5 and 17 define the palm plane. Tip landmarks are drawn in amber; the index tip — the one carrying the smoothed marker — is ringed in red.

2.2.1 The landmark table

The naming convention is systematic, and worth internalising: **MCP** is the knuckle (metacarpophalangeal joint), **PIP** the first finger joint, **DIP** the second, and **TIP** the end of the finger. **CMC** is the base of the thumb, at the wrist.

Index	Name	Short	Role in this project
0	Wrist	WRIST	Origin of both palm vectors; defines the palm plane
1	Thumb CMC	—	<i>(not used directly)</i>
2	Thumb MCP	—	<i>(not used directly)</i>
3	Thumb IP	THUMB_IP	The thumb rule’s “comparison” point
4	Thumb tip	THUMB_TIP	The thumb rule’s “extended?” point
5	Index MCP	INDEX_MCP	Second palm vector endpoint; index-finger base
6	Index PIP	—	The knuckle index 8 must beat vertically
7	Index DIP	—	<i>(not used directly)</i>
8	Index tip	INDEX_TIP	The accentuated, EMA-smoothed marker
9	Middle MCP	—	<i>(not used directly)</i>
10	Middle PIP	—	The knuckle index 12 must beat vertically
11	Middle DIP	—	<i>(not used directly)</i>
12	Middle tip	—	Middle finger extension test
13	Ring MCP	—	<i>(not used directly)</i>
14	Ring PIP	—	The knuckle index 16 must beat vertically
15	Ring DIP	—	<i>(not used directly)</i>
16	Ring tip	—	Ring finger extension test
17	Pinky MCP	PINKY_MCP	The thumb rule’s fixed reference point
18	Pinky PIP	—	The knuckle index 20 must beat vertically
19	Pinky DIP	—	<i>(not used directly)</i>
20	Pinky tip	—	Pinky finger extension test

Only nine of the twenty-one landmarks do any work. The project reads indices 0, 3, 4, 5, 8, 12, 16, 17, 20 — plus the four PIP joints 6, 10, 14, 18 as vertical references. That is a feature, not an oversight: a rule that uses fewer inputs is easier to reason about, easier to test, and less exposed to jitter in landmarks that carry no information.

2.2.2 Why 21 points, and not a bounding box or a mask

ML Concept 2.3 • The representation choice, and what it buys you

There are three common ways to represent a detected hand:

- **A bounding box** — four numbers. Cheap, but useless for fingers: a fist and an open hand have the same box.
- **A segmentation mask** — a pixel-per-pixel “is this hand?” image. Precise, but expensive, and it still does not tell you *which* pixel is a knuckle.
- **Keypoints** — a small, fixed set of semantically named points. Compact, cheap to compare, and every point has a *meaning*: index 8 *is* the index fingertip.

The project needs semantics, not pixels. Its questions are “is this fingertip above that knuckle?” and “how far is the thumb from the pinky knuckle?” — questions about *identified joints*. Keypoints answer them directly with 63 numbers (21×3) instead of a mask with 307,200.

2.3 The dependency matrix, and three package traps

The project pins exactly three packages:

Pin	Installs	Why exactly this
mediapipe==0.10.14	the model + runtime	The last release family with the legacy <code>mp.solutions.hands</code> API the project is built on. Everything from 0.10.30 onward has removed it.
opencv-contrib-python==4.11.0.86	cv2	The <i>contrib</i> build, not <code>opencv-python</code> : MediaPipe already depends on it, so pinning the same package avoids two competing <code>cv2</code> installs.
numpy<2	numerical arrays	MediaPipe 0.10.x wheels are compiled against the NumPy 1.x ABI. NumPy 2.0 changed that ABI.

Each of the three is a real trap that has cost real people real hours. Take them one at a time.

Edge-Case Trap 2.1 • Trap 1 — two cv2 packages fighting over one name

OpenCV ships under several PyPI names: `opencv-python`, `opencv-contrib-python`, `opencv-python-headless`, and so on. They all install the **same importable module name**, `cv2`, into site-packages.

MediaPipe’s own dependency list includes `opencv-contrib-python`. So this `requirements.txt` would be actively harmful:

```
1 mediapipe==0.10.14
2 opencv-python==4.10.0.84      # <- WRONG: a second, competing cv2
3 numpy<2
```

`pip` will happily install both. Then `import cv2` resolves to whichever package wrote its files last, and upgrading one can silently break the other — producing the notorious “it works on my machine” failures that are almost impossible to debug because `cv2.\_\_version\_\_` can report one thing while the code that runs comes from the other.

The fix. Pin the package MediaPipe already wants. The *contrib* build is a strict *superset* of the base build — it adds extra modules — so nothing is lost:

```
1 mediapipe==0.10.14
2 opencv-contrib-python==4.11.0.86
3 numpy<2
```

Edge-Case Trap 2.2 • Trap 2 — NumPy 2.0 and the ABI break

NumPy 2.0 (released mid-2024) changed its C application binary interface. Extension modules compiled against NumPy 1.x — which includes the MediaPipe 0.10.x wheels — can fail at import time with messages about binary incompatibility, or worse, import and then crash.

Pinning `numpy<2` resolves it to the 1.26.x line, which is also the last version supporting Python 3.9–3.12 across the board. If you see `pip` proposing to upgrade NumPy as a “helpful” side effect of another install, decline.

Edge-Case Trap 2.3 • Trap 3 — `mp.solutions` no longer exists (the load-bearing pin)

This is the one that matters most, because it is not a subtlety — it is a hard removal. The legacy API used by this project, `mp.solutions.hands.Hands`, was deprecated and then

removed in MediaPipe 0.10.30+. On a newer version the very first attribute access fails:

```
>>> import mediapipe as mp
>>> mp.solutions.hands
AttributeError: module 'mediapipe' has no attribute 'solutions'
```

Consequence: `pip install -U mediapipe` bricks the application. The pin `mediapipe==0.10.14` is therefore not a preference — it is a load-bearing structural element, in the same sense as a beam holding up a floor.

Defence. Rather than let the user discover this through an opaque `AttributeError` three imports deep, the application checks at import time and fails loudly with an actionable message. Here is the real guard, from `hand_tracker.py`:

```
1 if not hasattr(mp, "solutions") or not hasattr(mp.solutions, "hands"):
2     raise RuntimeError(
3         "mediapipe>=0.10.30 removed mp.solutions, which this app is built on. "
4         "Install the pinned version instead: pip install -r requirements.txt "
5         "(do NOT `pip install -U mediapipe`)."
6     )
```

- `hasattr(mp, "solutions")` probes the attribute *without* raising — the whole point of `hasattr` is that it returns `False` instead of blowing up. This is exactly the situation it was designed for.
- The `or` short-circuits: if `solutions` is missing entirely, the second `hasattr` is never evaluated, so the guard itself cannot crash.
- `raise RuntimeError` rather than `sys.exit` keeps the failure inside Python's normal error machinery, so a test harness or an editor sees a real traceback.
- The message names *the fix*, not just the problem. A good error message tells you what to type next.

2.3.1 Suppressing the deprecation noise

Deprecated APIs like these emit warnings on import, which pollute a live console that is already printing FPS telemetry. The project silences them *only around the import*, so genuine warnings elsewhere still surface:

```
1 with warnings.catch_warnings():
2     warnings.simplefilter("ignore")
3     import mediapipe as mp
```

The `with` block restores the previous warning behaviour on exit. Warning filters are global state; changing them permanently to hide one import's noise would suppress diagnostics you may actually need later — for instance a NumPy deprecation that only appears under load.

2.4 Confidence thresholds: what 0.7 and 0.6 actually control

The tracker constructs the model with four arguments. Two of them are thresholds, and they are the difference between a twitchy detector and a sluggish one.

```

1 self.hands = mp_hands.Hands(
2     static_image_mode=mode,
3     max_num_hands=max_hands,
4     min_detection_confidence=det_conf,      # default 0.7
5     min_tracking_confidence=track_conf,     # default 0.6
6 )

```

ML Concept 2.4 • Two thresholds, two different questions

- `min\_detection\_confidence = 0.7` applies when the system is *looking for a new hand* — stage 1, the palm detector running over the whole frame. A candidate box must score at least 0.7 to be accepted.
- `min\_tracking\_confidence = 0.6` applies when the system is *following a hand it already found*, using the landmarks predicted from the previous frame. The bar is lower because tracking has more evidence: it knows where the hand just was.

Why the asymmetry? Detection is a *false-positive* problem: if you accept weak candidates, a folded sleeve or a face can spawn a phantom hand. Tracking is a *false-negative* problem: if you drop the hand for one frame, the on-screen count flickers and the debouncer restarts. So the design is deliberately conservative about discovering hands (0.7) and forgiving about keeping them (0.6). Lowering detection to 0.5 is the usual first experiment for a dim room — with the warning that phantom hands become likelier. Raising it toward 0.9 makes the tracker refuse to see your hand until it is well lit and centred.

2.4.1 `static\_image\_mode` and `max\_num\_hands`

- `static\_image\_mode=True` disables temporal tracking. Every frame is treated as an unrelated still, so the detector runs every time. It is slower and noisier on video, but it is exactly right for `-image` mode, where there is only one frame and nothing to track. The CLI exposes it as `-static` for live use; `run\_image()` always sets it to `True`.
- `max\_num\_hands=2` caps how many hands are searched for. It is a genuine performance knob: the detector’s cost grows with the number of candidate boxes it keeps. Two is the right default for a two-handed demo; raise it only if you need more.

2.5 Handedness, and why the label inverts

The model also emits a **handedness** classification — “Left” or “Right” — with a score. Beginners assume this is a property of the hand in the image. It is not: it is a prediction made under an assumption about how the image was captured.

Edge-Case Trap 2.4 • The mirrored-feed trap: MediaPipe assumes a selfie

MediaPipe’s documentation is explicit: handedness is predicted *assuming the input image is mirrored*, as when a front-facing camera feed has already been flipped horizontally — the normal convention for a selfie.

This project flips the frame for exactly that reason — a mirrored view is what makes a webcam feel like a mirror and makes “move right” move right on screen:

```
frame = cv2.flip(frame, 1)
```

But that flip happens **before** MediaPipe ever sees the frame. The result: on many webcams and configurations the label comes back *inverted* — your physical right hand is reported as “Left”.

Why this is handled by design rather than patched. The project never lets the label drive geometry. The finger logic uses handedness only in the explicitly opt-in `-thumb-mode x`; the default thumb rule is handedness-free by construction (Section 3.2). The label is treated as *display telemetry*, and the escape hatch is a display-only fix:

```
-swap-handedness    # swap the Left/Right labels in the HUD
```

Because the swap happens inside `parse\_hand\_data`, it also flows through to the per-hand smoothing and debouncing keys, so no state is orphaned by the change.

Chapter summary. Detection is deliberately split in two: a cheap detector finds a palm box, and an expensive network regresses 21 named 3D landmarks inside that crop — which is what makes real-time performance on a CPU achievable. The three pins each defend against a specific, real failure: two competing cv2 packages, the NumPy 2 ABI break, and the removal of `mp.solutions`. The two confidence thresholds are tuned asymmetrically — strict about discovering hands, forgiving about losing them — and the handedness label is display-only, because on a mirrored feed it cannot be trusted.

Mathematical and Geometric Algorithms

Everything the camera sees has now become 21 points with names (Chapter 2). This chapter is the part of the project that is *ours*: turning those points into answers. It is the only chapter where the reasoning is original rather than borrowed from a library, and it is where every interesting bug lives.

The chapter moves from the simplest question to the most subtle:

1. Which way is “up” on a screen, and why does that invert the finger test? (§3.1)
2. Is the thumb out? — the rule that survives rotation, and the physical pose that still defeats it. (§3.2)
3. Which side of the hand is facing the camera? (§3.3)
4. How do we stop the numbers shaking? (§3.4)
5. What gesture is this, and how do we stop the label flickering? (§3.5)

3.1 The screen coordinate system, and the inverted y-axis

Math Deep Dive 3.1 • Two coordinate systems, one picture

In the mathematics you learned at school, the **Cartesian plane** has its origin in the middle or at the bottom-left, the x -axis grows to the right, and y **grows upward**. A point “higher up” has a *larger* y .

A computer screen inherited a different convention from the cathode-ray tubes of the 1960s: the electron beam scanned from the top-left corner, so that became the origin, and the beam’s downward sweep became the positive y direction. ****On a screen, y grows downward.**** A point higher up has a *smaller* y .

The two systems are related by a vertical flip:

$$y_{\text{cartesian}} = H - y_{\text{screen}}$$

where H is the image height in pixels. Nothing else changes; x grows to the right in both.

3.1.1 Why a fingertip being “above” its knuckle means a smaller y

This single fact is the whole basis of the four-finger test.

Math Deep Dive 3.2 • Deriving the finger test

Hold your hand up in front of the camera with the index finger extended. In the image, the fingertip is physically higher than the knuckle.

Because y grows *downward*, “physically higher” means “numerically smaller”:

$$y_{\text{tip}} < y_{\text{knuckle}} \iff \text{the fingertip is higher on screen}$$

Now bend the finger so the tip curls down toward the palm. The tip drops below the knuckle in the image, so its y becomes *larger*:

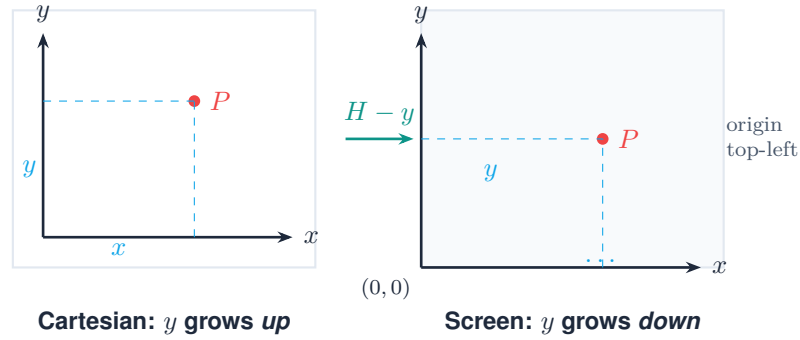


Figure 3.1. The same point P in the two conventions. In the Cartesian frame the vertical arrow points up and P has a positive y . On a screen the vertical arrow points down and the identical point now has y measured from the top-left corner. Every “my marker is mirrored vertically” bug is this figure being forgotten.

$$y_{\text{tip}} > y_{\text{knuckle}} \iff \text{the finger is folded}$$

The test is therefore:

$$\text{finger extended} \iff y_{\text{tip}} < y_{\text{pip}}$$

and the comparison is *strict* — an exactly equal y counts as folded, which costs nothing and keeps the rule total (it always returns a definite answer).

Worked example. Say the index fingertip is at pixel (346, 235) and the index PIP joint at (350, 280). Then $y_{\text{tip}} = 235 < y_{\text{pip}} = 280$, so the finger is extended — and indeed 45 pixels of difference is a clearly raised finger. If instead the tip were at (352, 305), then $305 > 280$ and the finger is folded.

The four long fingers are tested with four (tip, knuckle) index pairs from `hand_logic.py`:

```
1 FINGER_PAIRS = [(8, 6), (12, 10), (16, 14), (20, 18)]
```

Finger	Tip index	Knuckle (PIP) index	Test
Index	8	6	$y_8 < y_6$
Middle	12	10	$y_{12} < y_{10}$
Ring	16	14	$y_{16} < y_{14}$
Pinky	20	18	$y_{20} < y_{18}$

Edge-Case Trap 3.1 • The y -rule’s hidden assumption: fingers must point roughly up

The test compares y only. That is a comparison of *vertical* position, and it is only meaningful while the fingers point roughly toward the top of the image.

Point your index finger horizontally to the right (thumb up, like a finger-gun) and the tip and knuckle have almost the same y . The tip may even be slightly lower. The rule will report the finger as folded while it is plainly extended.

This is a genuine limitation, not a bug, and it is why the tracker is documented as a *upright-hand* tool. Making it fully rotation-invariant would require comparing the fingertip against the palm axis — computing the finger’s direction relative to the wrist-to-middle-knuckle vector and asking whether the tip is farther along that axis than the knuckle. That is a real technique, and the thumb rule below uses exactly that family of idea (a distance from

a fixed palm reference), but applying it to all four fingers is more machinery than this project needs.

Practical consequence for you as a user: keep your hand upright. **Practical consequence as an engineer:** if you extend this project, this is the first rule to replace.

3.2 The thumb, and the search for invariance

The thumb is the hard one. It moves on a different axis from the other four fingers, it is foreshortened by perspective when it points at the camera, and its apparent left-right position depends on which hand it is *and* which way the palm is facing.

3.2.1 The obvious rule, and exactly how it breaks

The original specification proposed a horizontal comparison. Because a webcam feed is mirrored, and because MediaPipe’s handedness label is predicted under a mirrored-feed assumption, the rule was written as:

Right $\Rightarrow$ thumb extended if $x_{\text{tip}} < x_{\text{ip}}$

Left $\Rightarrow$ thumb extended if $x_{\text{tip}} > x_{\text{ip}}$

Edge-Case Trap 3.2 • Why the horizontal rule fails: palm inversion

The rule silently encodes a physical assumption — *the palm faces the camera*. Turn your hand around so the back faces the camera, and everything the rule knows becomes false. Concretely: hold up your right hand, palm toward the camera, thumb out. The thumb points to one side of the image. Now rotate your wrist 180° so the back of the hand faces the camera, keeping the thumb extended. The thumb is now on the *other* side of the image, while the hand is still your right hand and the label still reads “Right”. The comparison $x_{\text{tip}} < x_{\text{ip}}$ now returns the opposite answer.

The result: the finger count flickers as you rotate your hand, and the gesture classification collapses with it. Rotation of the wrist alone — no change in pose at all — flips the answer. There is a second, subtler failure: the comparison is purely horizontal, so a hand rotated in the image plane (say, 45°) has a thumb whose $x_{\text{tip}} - x_{\text{ip}}$ difference is scaled by $\cos 45^\circ \approx 0.71$, shrinking the margin and making the test fragile near the decision boundary.

The rule is kept in the code because the specification asked for it, reachable as `-thumb-mode x`, but it is not the default.

3.2.2 The distance rule

The idea in one sentence. However the hand is rotated, an **extended thumb reaches away from the palm**, while a folded thumb is tucked *across* the palm — so measure how far the thumb tip and the thumb joint sit from a fixed palm reference, and see which is farther.

The fixed reference is landmark 17, the pinky MCP — the far corner of the palm, the point the thumb moves *toward* when it folds and *away* from when it extends.

Math Deep Dive 3.3 • The rule, derived and justified

$$\text{thumb extended} \iff d(P_4, P_{17}) > d(P_3, P_{17})$$

where P_4 is the thumb tip, P_3 is the thumb IP joint, P_{17} is the pinky MCP, and d is the ordinary Euclidean distance

$$d(A, B) = \sqrt{(A_x - B_x)^2 + (A_y - B_y)^2}.$$

Why a distance and not a coordinate. A coordinate is tied to the frame of reference: x means “how far right on the screen”, which changes when the hand rotates. A distance is an *intrinsic* quantity — it depends only on the two points themselves.

The invariance argument, stated precisely. Suppose the hand is rotated in the image plane by an angle θ about any centre C . The rotation matrix is

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}, \quad R_\theta^\top R_\theta = I.$$

Every landmark is transformed as $P' = C + R_\theta(P - C)$. Therefore the difference $P'_4 - P'_{17} = R_\theta(P_4 - P_{17})$, and

$$\begin{aligned} d(P'_4, P'_{17}) &= \|R_\theta(P_4 - P_{17})\| \\ &= \sqrt{(P_4 - P_{17})^\top R_\theta^\top R_\theta (P_4 - P_{17})} \\ &= \sqrt{(P_4 - P_{17})^\top (P_4 - P_{17})} = d(P_4, P_{17}). \end{aligned}$$

The distance is *exactly* unchanged by any rotation, and the same holds for $d(P_3, P_{17})$. Since both sides of the comparison are preserved, the comparison’s result cannot change. The edge case is equality — the two distances are equal exactly when the thumb tip and IP joint lie on a circle centred on the pinky knuckle, which happens only in a degenerate straight-thumb pose; the strict $>$ then reports “folded”, which is the safe answer for a thumb that is not clearly away from the palm.

Handedness-independence. Nothing in the expression refers to “left” or “right”. It is a statement about three points, so it cannot be affected by a label that may itself be wrong.

Math Deep Dive 3.4 • Worked example, with numbers

This is the actual synthetic hand from the project’s test suite (a right hand, palm to camera, fingers up, mirrored view).

Thumb extended

$$\begin{aligned} P_4 &= (430, 320) && \text{(thumb tip)} \\ P_3 &= (415, 345) && \text{(thumb IP)} \\ P_{17} &= (262, 335) && \text{(pinky MCP)} \end{aligned}$$

$$\begin{aligned} d(P_4, P_{17}) &= \sqrt{(430 - 262)^2 + (320 - 335)^2} \\ &= \sqrt{168^2 + (-15)^2} = \sqrt{28224 + 225} = \sqrt{28449} \approx 168.7 \end{aligned}$$

$$\begin{aligned} d(P_3, P_{17}) &= \sqrt{(415 - 262)^2 + (345 - 335)^2} \\ &= \sqrt{153^2 + 10^2} = \sqrt{23409 + 100} = \sqrt{23509} \approx 153.3 \end{aligned}$$

Since $168.7 > 153.3$, the thumb is reported **extended**. ✓

Thumb folded. Now the tip folds across the palm to $P_4 = (375, 350)$, with the IP joint at $P_3 = (395, 355)$:

$$\begin{aligned} d(P_4, P_{17}) &= \sqrt{113^2 + 15^2} = \sqrt{12769 + 225} = \sqrt{12994} \approx 114.0 \\ d(P_3, P_{17}) &= \sqrt{133^2 + 20^2} = \sqrt{17689 + 400} = \sqrt{18089} \approx 134.5 \end{aligned}$$

Now $114.0 < 134.5$, so the thumb is reported **folded**. ✓

Notice the sizes involved: the margins are roughly 15 pixels and 20 pixels respectively. That is a comfortable margin at 640×480 — large enough that landmark jitter of one or two pixels cannot flip the answer.

3.2.3 The knife hand: when the distance rule lies

Edge-Case Trap 3.3 • The adducted-thumb false positive

Hold your hand flat, fingers straight, thumb pressed tightly against the side of the index finger — the “karate chop” or “knife hand”.

The thumb tip is physically at the far end of the hand, near the index fingertip, while the thumb IP joint is back near the palm. Distance to the pinky knuckle (landmark 17): the tip really *is* farther away than the IP joint. So the rule reports the thumb as **extended**, and the finger count is one too high.

This is not a coding error; it is the rule doing exactly what it says. The failure is that the rule only ever asks “is the thumb away from the pinky side of the palm?”, and in a knife hand the answer is genuinely yes.

The candidate fix — a secondary clearance gate against the index MCP (landmark 5), which the thumb must also be distant from:

$$\text{dist}(P_4, P_5) > 1.2 \cdot \text{dist}(P_3, P_5)$$

The factor 1.2 encodes “the tip must be *clearly* farther from the index knuckle than the IP joint, not merely farther”. In a knife hand the tip sits close to the index finger, so this second test fails and the false positive is suppressed.

Why this project does not enable it by default. The gate trades one error for another. On a thumb pointing sideways across the image — a genuine “thumbs up” rotated 90° by wrist tilt — the tip can pass close to the index knuckle and the gate would suppress a *correct* extended result. The factor 1.2 is a tuned constant with no derivation, and it varies with hand size and camera distance because it compares raw pixel distances.

The engineering position taken here: ship the rule that is rotation-invariant and documented, document the pose that defeats it, and leave the tuning to whoever has the hardware in front of them. The limitation is stated plainly in the project’s README rather than hidden.

3.2.4 Keeping the old rule reachable, and what happens with no label

The code implements both rules, with the distance rule as the default and a defined behaviour when the handedness label is unavailable:

```

1 if thumb_mode == THUMB_X and hand_label in ("Left", "Right"):
2     tip, ip = pixel_landmarks[THUMB_TIP], pixel_landmarks[THUMB_IP]
3     thumb = tip[0] < ip[0] if hand_label == "Right" else tip[0] > ip[0]
4 else:
5     thumb = _distance(pixel_landmarks[THUMB_TIP], pixel_landmarks[PINKY_MCP]) > _distance(
6         pixel_landmarks[THUMB_IP], pixel_landmarks[PINKY_MCP]
7     )

```

- The condition is a conjunction: the naive rule needs *both* `thumb_mode == "x"` and a usable label.
- If the label is "Unknown" — which happens when MediaPipe's handedness list is missing or short — the else branch applies **the distance rule**, not False. A missing label degrades to a working rule rather than silently losing the thumb.
- `tip[0]` is the *x* component of the tip tuple; `tip[1]` would be *y*. Keeping coordinate access as indexed tuples rather than named objects is what makes this expression short.

3.3 The palm normal: which side of the hand am I seeing?

Finger counting does not need this. It is included because it is exactly the kind of diagnostic telemetry the specification asks for, and because it makes the thumb rule's invariance visible while the program runs.

Math Deep Dive 3.5 • Two vectors and a cross product

Take two vectors from the wrist P_0 :

$$\vec{v}_1 = P_5 - P_0 \quad (\text{wrist} \rightarrow \text{index knuckle})$$

$$\vec{v}_2 = P_{17} - P_0 \quad (\text{wrist} \rightarrow \text{pinky knuckle})$$

In 3D, the palm's normal vector is their cross product $\vec{v}_1 \times \vec{v}_2$. Writing $\vec{v}_1 = (a_x, a_y, a_z)$ and $\vec{v}_2 = (b_x, b_y, b_z)$:

$$\vec{v}_1 \times \vec{v}_2 = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

The project uses **only the *z* component**, because the landmarks' *x* and *y* are pixel coordinates while *z* is relative depth — mixing them would be meaningless. With $a = (x_5 - x_0, y_5 - y_0)$ and $b = (x_{17} - x_0, y_{17} - y_0)$:

$$\begin{aligned} Z_{\text{normal}} &= a_x b_y - a_y b_x \\ &= (x_5 - x_0)(y_{17} - y_0) - (y_5 - y_0)(x_{17} - x_0) \end{aligned}$$

This is the signed area (times two) of the triangle wrist–index–knuckle–pinky–knuckle. Its **sign** records the rotational order of the two vectors — which way you turn going from $\vec{v}_1$ to $\vec{v}_2$ — and the two sides of a hand produce opposite orders.

Worked example. Using the test suite’s synthetic palm-facing hand $P_0 = (320, 420)$, $P_5 = (355, 320)$, $P_{17} = (262, 335)$:

$$\begin{aligned} Z &= (355 - 320)(335 - 420) - (320 - 420)(262 - 320) \\ &= (35)(-85) - (-100)(-58) \\ &= -2975 - 5800 = -8775 \end{aligned}$$

A *negative* Z for a palm facing the camera. Mirror the same hand horizontally (which is exactly what “showing the back of your hand” does to the knuckle ordering) and every x difference changes sign, so Z becomes $+8775$ — the sign flips, as it must.

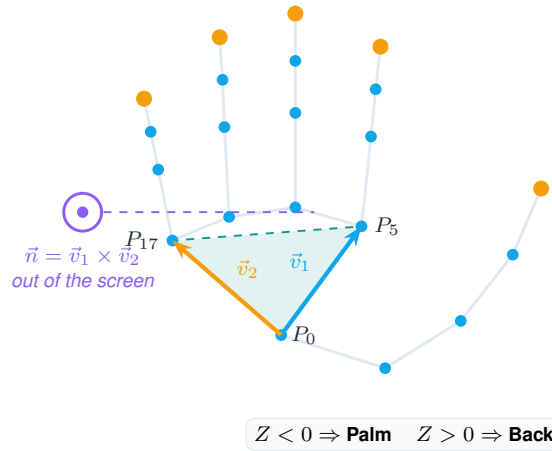


Figure 3.2. The palm normal. The wrist and the two outer knuckles define a plane; the sign of the z -component of $\vec{v}_1 \times \vec{v}_2$ says whether that plane is presented palm-first or back-first. Because the calculation uses only two differences, it is one subtraction cheaper than a true 3D cross product and needs no depth data.

3.3.1 The sign convention, and how to remember it

The sign-to-label mapping depends on the mirroring of the image, so the project states its convention explicitly: **a mirrored (selfie) view, as produced by `cv2.flip(frame, 1)`.**

Math Deep Dive 3.6 • Where the sign comes from

Consider a right hand, palm to camera, fingers up, in a horizontally mirrored image.

- In a mirrored view your right hand appears on the *right* side of the picture, and its thumb — pointing away from the palm — appears on the *far right* of the hand.
- Therefore the index knuckle P_5 lies to the *right* of the wrist, and the pinky knuckle P_{17} lies to the *left*: $x_5 > x_0$ and $x_{17} < x_0$.
- Both knuckles are *above* the wrist, and y grows downward, so $y_5 < y_0$ and $y_{17} < y_0$.

Substituting these signs into the formula from the previous box, $a_x b_y = (+)(-)$ and $a_y b_x = (-)(-)$, gives $Z = (\text{negative}) - (\text{positive}) < 0$.

Now turn the hand over. Index and pinky knuckles swap sides, so $x_5 < x_0$ and $x_{17} > x_0$; both y relations are unchanged. Now $a_x b_y$ is negative and $a_y b_x$ is positive, giving $Z > 0$. Hence, for the mirrored convention: $Z < 0 \Rightarrow$ **Palm facing**, $Z > 0 \Rightarrow$ **Back facing**. The project encodes exactly this:

```
def orientation_from_palm_z(palm_z):
    """Map the palm-normal sign to the HUD label ``Palm`` or ``Back``."""
    return "Palm" if palm_z < 0 else "Back"
```

The degenerate case $Z = 0$ (wrist, index knuckle and pinky knuckle collinear — geometrically impossible for a real hand, but reachable with synthetic or corrupted data) falls to Back through the else. A test asserts this, so the behaviour is fixed rather than accidental.

Edge-Case Trap 3.4 • The convention is tied to the mirror, not to the hand

Because the sign depends on the image being flipped, the label inverts in `-image` mode, where a still photograph is deliberately *not* mirrored. A photograph of a palm-first hand in `-image` mode will report Back.

That is not a bug in the geometry; it is the geometry faithfully reporting an unmirrored image with a mirrored-image convention. It is documented in the README, and it matters only for the diagnostic label — no finger decision depends on it.

3.4 Smoothing: the exponential moving average

Math Deep Dive 3.7 • Sensor jitter, and why raw coordinates shake

Even with a completely still hand, the reported landmark positions move by one to three pixels from frame to frame. The causes are mundane and unavoidable:

- camera sensor noise, especially in low light;
- micro-movements of your hand that you cannot consciously control;
- the model’s own regression error, which varies slightly as the hand’s appearance in the crop changes.

At 30 FPS, a dot drawn from raw coordinates therefore vibrates visibly — it reads as “cheap program”. The fix is a filter: blend each new measurement with the previous smoothed value.

$$\mathbf{P}_{\text{smooth}}^{(t)} = \alpha \mathbf{P}_{\text{raw}}^{(t)} + (1 - \alpha) \mathbf{P}_{\text{smooth}}^{(t-1)}, \quad \alpha = 0.65$$

Applied independently to x and y . The name is **exponential moving average** (EMA), or single-pole low-pass filter.

Why it works. Expand the recurrence backwards:

$$\begin{aligned} \mathbf{P}^{(t)} &= \alpha \mathbf{R}^{(t)} + (1 - \alpha) [\alpha \mathbf{R}^{(t-1)} + (1 - \alpha) \mathbf{P}^{(t-2)}] \\ &= \alpha \mathbf{R}^{(t)} + \alpha(1 - \alpha) \mathbf{R}^{(t-1)} + (1 - \alpha)^2 \mathbf{P}^{(t-2)} \\ &= \alpha \sum_{k=0}^n (1 - \alpha)^k \mathbf{R}^{(t-k)} + (1 - \alpha)^{n+1} \mathbf{P}^{(t-n-1)} \end{aligned}$$

The current position is a weighted sum of *all* past measurements, with geometrically decaying weights $\alpha, \alpha(1 - \alpha), \alpha(1 - \alpha)^2, \dots$: recent frames matter most, old frames fade out but never vanish entirely.

The noise-reduction price. The weights sum to 1 in the limit, so a constant true position is reproduced exactly. Random noise, however, is averaged down: the effective averaging window has length roughly $1/\alpha \approx 1.5$ frames of “memory mass” concentrated

in recent frames, and the residual jitter is reduced by roughly $\sqrt{\alpha/(2-\alpha)} \approx 0.69$ for $\alpha = 0.65$.

The responsiveness price. If the true position jumps (you flick your finger), the displayed value approaches it geometrically and never exactly arrives. After one frame it has covered $\alpha = 65\%$ of the gap; after two, $1 - (1 - \alpha)^2 = 87.75\%$; after three, 95.7% . Visible lag at 30 FPS is therefore on the order of one to two frames — about 30–60 milliseconds, below the threshold at which a user perceives the marker as “slow”.

Math Deep Dive 3.8 • Worked example, and the choice of α

Suppose the smoothed value is $P^{(t-1)} = 300$ and the new raw measurement is $R^{(t)} = 340$.

$$P^{(t)} = 0.65 \times 340 + 0.35 \times 300 = 221 + 105 = 326$$

So the dot moves 26 pixels of the 40-pixel gap — most of the way, but not all of it. The remaining 14 pixels will be consumed over the following frames.

Why $\alpha = 0.65$ rather than something else?

- $\alpha \rightarrow 1$ (say 0.95): almost no smoothing. Jitter is preserved; the dot is a raw measurement with a hat on.
- $\alpha \rightarrow 0$ (say 0.1): heavy smoothing. The dot becomes a lazy, laggy ghost that visibly trails the finger.
- $\alpha = 0.65$: covers 65% of any gap per frame — fast enough to feel instantaneous at video rates, slow enough to flatten single-frame noise.

First sighting. On the very first frame there is no previous value, so the recurrence has no starting point. Blending against zero would make the dot animate in from the top-left corner, which looks like a bug. Instead, the implementation treats “no previous” as a special case and passes the measurement through unchanged:

```
def ema(previous, current, alpha=0.65):
    if previous is None:
        return (current[0], current[1])
    return (
        alpha * current[0] + (1 - alpha) * previous[0],
        alpha * current[1] + (1 - alpha) * previous[1],
    )
```

Edge-Case Trap 3.5 • Smoothing state is keyed by hand, not by list index

MediaPipe does not guarantee the order of its hand list between frames. If your right hand leaves the frame, or the two hands cross, the hand at index 0 in one frame may be the hand at index 1 in the next.

If the smoothing history were stored per *index* — `\_smooth\_tip[0]`, `\_smooth\_tip[1]` — a reorder would blend the position of one physical hand with the history of the other. The visible symptom is a sudden teleport of the marker, which is precisely the behaviour the filter exists to prevent.

The project keys the state by **handedness label** instead, falling back to the index only when the label is unavailable:

```
@staticmethod
def _state_key(hand_index, label):
    return label if label in ("Left", "Right") else f"index:{hand_index}"
```

This is not perfect — handedness itself can flicker — but it is meaningfully more stable than a raw index, and it costs nothing. The same key is reused for the gesture debouncer, so a reorder cannot contaminate gesture history either.

3.5 Gestures and the debouncer

3.5.1 From five booleans to a name

The finger logic produces five booleans in a fixed order — [thumb, index, middle, ring, pinky]. A gesture is simply a recognised pattern in those five bits.

Gesture	Thumb	Index	Middle	Ring	Pinky	Notes
Open Palm	✓	✓	✓	✓	✓	all five extended
Fist						all five folded
Thumbs Up	✓					thumb alone
Peace		✓	✓			the V
Rock On	✓	✓			✓	index, pinky, thumb
(anything else)	—	—	—	—	—	reported as —

The order of the checks in the code matters, and is written so that the more specific patterns are tested first. Note that “Open Palm” (all five true) and “Thumbs Up” (only the thumb true) cannot collide, but reading the code top to bottom shows the intent:

```

1 if thumb and index and middle and ring and pinky:
2     return "Open Palm"
3 if not (thumb or index or middle or ring or pinky):
4     return "Fist"
5 if thumb and not (index or middle or ring or pinky):
6     return "Thumbs Up"
7 if not thumb and index and middle and not (ring or pinky):
8     return "Peace"
9 if thumb and index and not (middle or ring) and pinky:
10    return "Rock On"
11 return NO_GESTURE

```

The negated conditions (“not ring and not pinky”) are what make each pattern exact rather than a prefix match. Without them, “Peace” would also match a hand with all five fingers up.

3.5.2 Why a raw per-frame answer flickers

Math Deep Dive 3.9 • Temporal majority voting

During a transition — say from Fist to Peace — the fingers do not move in perfect synchrony, and for a few frames the pattern belongs to no gesture at all. Classifying each frame independently produces a label that flashes through intermediate states. Worse, a single noisy frame in the middle of a steady pose can produce a one-frame spike to a wrong label.

The fix is a **rolling majority vote** over a short history. Formally, with window length n and required agreement k , at frame t :

$$\text{report } g \iff \sum_{i=t-n+1}^t \mathbb{I}[g_i = g] \geq k, \quad n = 5, k = 4$$

where $\mathbb{I}[\cdot]$ is the indicator function (1 if the condition holds, 0 otherwise). If no label reaches k , the previously reported label is held.

The implementation is a deque with `maxlen=5`, which gives the sliding window for free, and a count on the newest label:

```
def update(self, gesture):
    self._history.append(gesture)
    if self._history.count(gesture) >= self.min_agree:
        self._stable = gesture
    return self._stable
```

Worked trace. The window is the last five entries; entries are listed oldest first.

Frame	Input	Window after append	Returned
1	Peace	P	–
2	Peace	P P	–
3	Peace	P P P	–
4	Peace	P P P P	Peace
5	Peace	P P P P P	Peace
6	Fist	P P P P F	Peace
7	Fist	P P P F F	Peace
8	Fist	P P F F F	Peace
9	Fist	P F F F F	Fist

Reading it carefully: a change of gesture takes exactly four frames to be accepted, and a two-frame blip can never break through, because two frames cannot reach a count of four in a five-slot window. That is the precise trade: ≈ 130 ms of latency at 30 FPS, in exchange for the complete elimination of single- and double-frame flicker.

Warm-up behaviour is defined, not accidental. Before any majority has formed, the stable label is the placeholder –, so the HUD shows no gesture for the first three frames of a session rather than guessing. The alternative — returning the first raw label immediately — would make the debouncer pointless on startup, since the first frame is exactly the one most likely to be noisy.

3.6 The complete decision path

Putting the chapter together, one frame produces this chain, entirely inside `hand\_logic.py`:



Figure 3.3. The per-frame decision chain in the pure-logic module. Every stage is a plain function of its inputs — no global state, no camera, no I/O — which is why the whole chain is unit-testable with hand-written coordinates.

Chapter summary. On a screen y grows downward, so a fingertip is extended when $y_{\text{tip}} < y_{\text{pip}}$ for the four long fingers. The thumb uses a distance comparison against the pinky knuckle, which is provably invariant under image-plane rotation and independent of handedness — with the adducted “knife hand” documented as its known failure. The palm normal’s sign reveals which side of the hand faces the camera. An exponential moving

average with $\alpha = 0.65$ removes jitter at the cost of roughly a frame of latency, and a five-frame, four-vote rolling majority removes gesture flicker at the cost of roughly four frames of latency.

System Design and Production Architecture

The algorithms are settled. This chapter is about everything that makes them survive contact with a real machine: how the code is split, why there is a cache, what happens when the camera is missing, how the whole thing is tested without hardware, and how a text overlay is drawn so that it stays readable.

4.1 Architectural separation

The project is two modules, and the boundary between them is the most important design decision in the codebase.

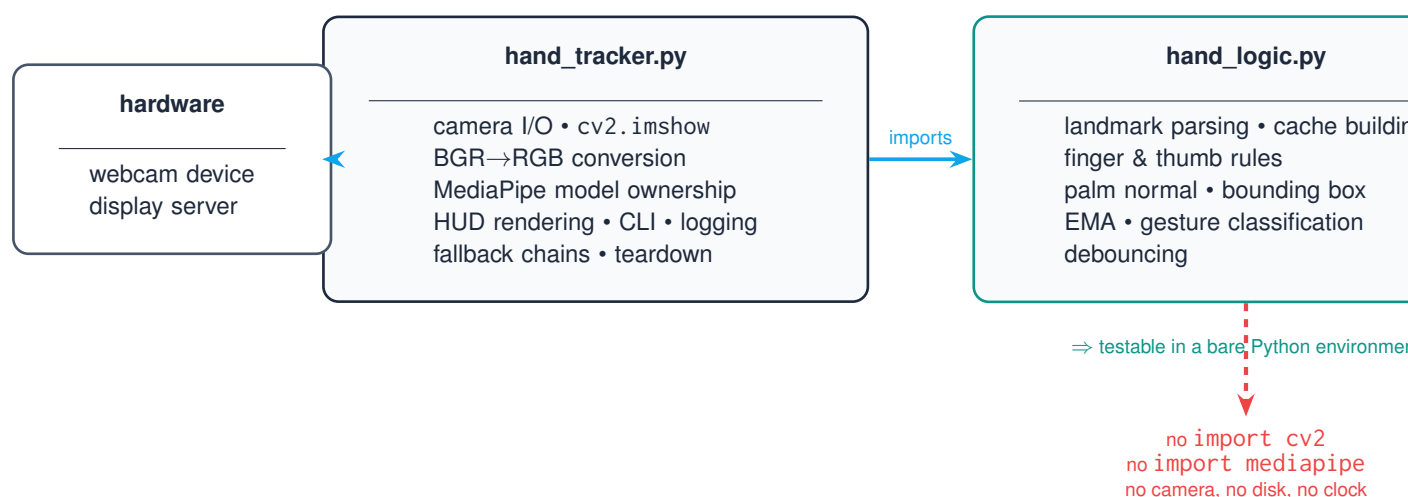


Figure 4.1. The dependency rule. All hardware, all third-party libraries and all I/O live on the left. The right-hand module is pure: it can be imported, exercised and exhaustively tested in a bare Python interpreter with no camera, no display and no OpenCV installed. The dashed red arrow states what must never appear in it.

4.1.1 Why the split exists

The original specification asked for a single `hand_tracker.py` containing everything, with “unit-testable helpers”. Those two requirements contradict each other: a helper that lives in a file which imports `cv2` and `mediapipe` cannot be tested without installing both, and importing the module to test one geometry function drags in a 40-megabyte native library and, on a headless machine, may fail outright.

Splitting resolves the contradiction cleanly:

- `hand_logic.py` imports `math` and `collections` only. Its functions take plain numbers and lists and return plain numbers and lists.
- `hand_tracker.py` owns everything that touches the outside world.

The rule, stated as a test you can apply to any new line of code: *would this line work if there were no camera, no screen and no MediaPipe?* If yes, it belongs in `hand\_logic.py`. If no, it belongs in `hand\_tracker.py`.

Everything the project knows about *hands* is on the pure side. Everything it knows about *computers* is on the other.

4.1.2 What the split buys, concretely

Benefit	Consequence in this project
Fast tests	The logic suite runs in under a second, with no model loading and no hardware.
CI without a GPU	The continuous-integration job needs <code>pip install pytest</code> and nothing else — no 200 MB wheel, no display server.
Deterministic tests	Synthetic coordinates produce exactly predictable answers; a real webcam never would.
Portability	The geometry is valid on any platform, at any resolution, for any camera.
Debuggability	When the count is wrong you can ask “is the geometry wrong, or is the input wrong?” and answer it by feeding the geometry a known-good point set.

The test suite makes the payoff visible: the majority of the tests construct arrays of coordinates by hand and assert on booleans — no hardware anywhere in sight.

4.2 Single-pass telemetry caching

4.2.1 The problem: repeated work in a hot loop

A naive implementation converts normalized landmarks to pixels wherever it needs them. In one frame, the code needs pixel coordinates for: the index-tip marker, all five finger tests (six landmarks), the thumb rule (three landmarks), the palm normal (three landmarks), and the bounding box (all 21). Recomputing inside each of those means iterating the landmark list five or more times per hand, per frame, at 30 frames per second — thousands of redundant multiplications per second.

That is not a catastrophe. It is waste, and waste in a real-time loop is what separates a program that holds 30 FPS on a modest laptop from one that drops frames under load.

4.2.2 The solution: one parse, one dictionary per hand

```

1  hands.append(
2      {
3          "label": label,
4          "norm": norm,
5          "px": px,
6          "bbox": bbox_from_pixels(px),
7          "palm_z": z,
8          "orientation": orientation_from_palm_z(z),
9      }
10 )

```

Everything downstream reads from this structure. The cost of the conversion is paid exactly once per hand per frame.

Key	Type	What it stores, and who consumes it
label	str	"Left", "Right" or "Unknown", after any swap_handedness. Shown in the HUD; used as the state key for smoothing and debouncing.
norm	list[tuple]	The raw (x, y, z) triples in $[0, 1]$. Kept because it is the model's own output — useful for debugging, and free to retain.
px	list[tuple]	Integer pixel landmarks. Consumed by every geometric rule and by the tip marker.
bbox	tuple	$(x_{\min}, y_{\min}, x_{\max}, y_{\max})$. The hand's screen extent; used for labelling and diagnostics.
palm_z	float	The signed palm-normal value from Chapter 3.
orientation	str	"Palm" or "Back" — the label derived once from palm_z, so no consumer has to repeat the sign test.

Edge-Case Trap 4.1 • The cache has a lifetime, and it is one frame

find_positions is called with a frame argument, but it never reads that argument — the pixels were computed inside the preceding find_hands call, using *that* frame's dimensions. The parameter exists only to preserve the signature the specification asked for.

The consequence is a real constraint: **the cached pixel coordinates are only valid for a frame of the size they were computed from**. Reuse the tracker against a differently sized frame without calling find_hands first and the coordinates will be scaled wrongly — the dot will be drawn in the wrong place, with no error raised.

The rule to follow: one find_hands per frame, then read. Never cache across frames, never mix sizes. This is documented in the method's docstring precisely because the failure is silent.

4.2.3 Invalid input, and the safe result

Parsing is defensive because the model's output shape can vary: a hand with fewer than 21 landmarks cannot be reasoned about, and a missing handedness list is normal in some configurations.

```

1 hand_landmarks = getattr(results, "multi_hand_landmarks", None)
2 if not hand_landmarks:
3     return []
4
5 handedness = getattr(results, "multi_handedness", None) or []
6 ...
7 landmarks = getattr(hand, "landmark", None)
8 if not landmarks or len(landmarks) < EXPECTED_LANDMARKS:
9     continue

```

Three defensive decisions, each with a reason:

- `getattr(..., None)` rather than `results.multi\_hand\_landmarks`: a result object with no hands may legitimately lack the attribute. `getattr` returns the default instead of raising `AttributeError`.
- `... or []` handles the “attribute exists but is `None`” case, which would otherwise break `len()`.
- `continue` rather than `break` for a short landmark list: one malformed hand does not discard the other, valid hand in the same frame.

The same defensiveness applies to the finger rules, which return `[False] * 5` — never raise,

never return None — when handed nothing usable. That contract is asserted by a test, so callers may rely on it.

4.3 Camera fallback and hardware resiliency

Opening a webcam is the least reliable operation in the entire program. The device may be absent, in use by a video call, or present but only on a different index. A single-shot `cv2.VideoCapture(0)` fails on all three.

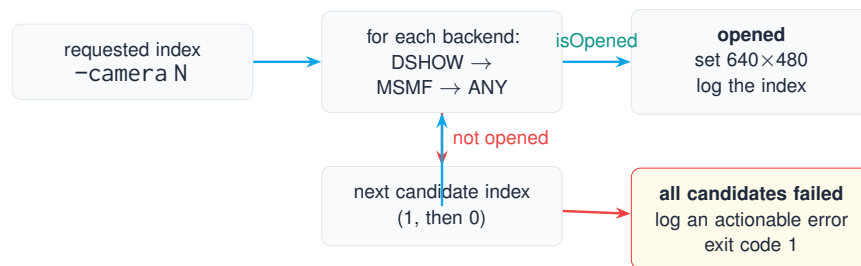


Figure 4.2. The camera fallback chain. Both axes are searched — backends within one index, and then other indices — before giving up with a message that names the likely cause.

```

1 for candidate in [index] + [other for other in (1, 0) if other != index]:
2     for backend in backends:
3         capture = cv2.VideoCapture(candidate, backend)
4         if capture.isOpened():
5             capture.set(cv2.CAP_PROP_FRAME_WIDTH, width)
6             capture.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
7             LOG.info("Camera %d opened at %dx%d.", ...)
8             return capture
9         capture.release()
10        LOG.warning("Camera %d did not open.", candidate)
11
12 LOG.error("Could not open a webcam. Check it is connected and not in use by another app.")
13 return None
  
```

- **Backends differ per platform.** On Windows the list is `[CAP_DSHOW, CAP_MSMF, CAP_ANY]`: DirectShow is the most permissive for consumer webcams, Media Foundation is the modern stack, and `CAP_ANY` lets OpenCV choose. Elsewhere the list is just `[CAP_ANY]`, because the alternatives do not exist.
- **The candidate list is `[index] + [1, 0]`** with the requested index removed, so `-camera 0` tries 0 then 1, while `-camera 5` tries 5, then 1, then 0.
- `capture.release()` on every failure is not optional. Without it, each failed attempt leaks a device handle, and on Windows a leaked handle can itself prevent the next attempt from succeeding — the fallback chain would sabotage itself.
- **Width and height are requested, not guaranteed.** `cap.set` is a request to the driver; the log line reports what was actually granted, which is why it re-reads the properties rather than echoing the requested numbers.

Edge-Case Trap 4.2 • “Could not open a webcam” — the four real causes

The error text is deliberately specific because the causes are mundane and diagnosable:

1. The camera is in use by another application (a video call, a browser tab holding the

device, or a previous run of this program that was killed without cleanup).

2. The index is wrong — common on laptops with both an integrated and an external camera, and on desktops where a capture card occupies index 0.
3. Operating-system privacy settings are blocking camera access for desktop applications.
4. The device is genuinely absent or disabled in Device Manager.

Cause 1 is the most common by a wide margin, and it is the reason teardown matters (Section 4.6).

4.4 Headless mode: verifying without a webcam

Every test in the project’s logic suite passes without a camera — but until this feature existed, *the drawing and rendering path* could only be exercised by a human sitting in front of a webcam. That is a large untested surface: the mesh drawing, the tip marker, the HUD blending, the CLI, the image encoding.

`-image PATH` closes that gap. It runs the entire pipeline — load, detect, parse, compute, draw, write — on a single still.

Step	Call	Purpose
1	<code>cv2.imread(path)</code>	Returns None on failure; the code checks and exits non-zero.
2	<code>cv2.resize</code>	Only if <code>-width</code> and <code>-height</code> were supplied. A still has a native resolution worth preserving.
3	<code>HandTracker(mode=True)</code>	<code>static_image_mode</code> — there is no next frame to track into.
4	<code>find_hands</code> → <code>annotate_hands</code> → <code>draw_hud</code>	<i>The same functions the live loop calls.</i> No parallel implementation exists to drift out of sync.
5	<code>cv2.imwrite</code>	Writes <code><stem>_out.png</code> next to the input, then exits 0.

Note the deliberate omission: the still is **not mirrored**. A photograph is not a selfie, and flipping it would move every landmark the model found to the wrong side of the image.

4.4.1 Continuous integration without a display

```
name: CI
on: [push, pull_request]

jobs:
  logic-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "${{ matrix.python-version }}" }
      - run: pip install pytest
```

```

- run: pytest -q

pipeline-smoke:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.12" }
    - run: sudo apt-get update && sudo apt-get install -y libgl1 libgl1-mesa-glx
    - run: pip install -r requirements.txt pytest
    - run: pytest -q
    - run: python hand_tracker.py -image tests/fixtures/hand.png
    - run: test -f tests/fixtures/hand_out.png

```

Edge-Case Trap 4.3 • libGL.so.1: the OpenCV-on-CI gotcha

The pipeline-smoke job installs the real dependencies — including `opencv-contrib-python`, which is *not* the headless build. Those wheels link against system graphics libraries that a minimal container does not have, and the failure is a stark

```
ImportError: libGL.so.1: cannot open shared object file: No such file or directory
```

It looks like a broken install or a code bug. It is neither: it is a missing shared library in the runner image. Installing `libgl1` (and `libgl1-mesa-glx`, which the same wheels frequently need) fixes it in one line. This is the single most commonly hit OpenCV/CI failure, which is why it is commented in the workflow itself rather than left as folklore.

4.5 The alpha-blended HUD

Drawing white text directly onto a webcam image fails the moment anything bright is behind it — a window, a lamp, a white wall. The fix is to darken a rectangle behind the text before writing it.

Math Deep Dive 4.1 • Alpha blending

For each pixel in the card region, the output is a weighted average of what was there and the overlay:

$$I_{\text{out}} = (1 - \alpha) I_{\text{roi}} + \alpha I_{\text{overlay}}$$

The project uses $\alpha = 0.6$ with a black overlay:

$$I_{\text{out}} = 0.4 I_{\text{roi}} + 0.6 \times 0 = 0.4 I_{\text{roi}}$$

So a bright background pixel of 255 becomes 102 — comfortably dark — while a dark background of 40 becomes 16. The result is that the card is visible as a subtle darkening on any background, and white text on top is readable at any luminance. Note that the formula is applied *per channel*, so hues are preserved and only brightness is scaled.

```

1 card = frame[y0:y1, x0:x1]
2 frame[y0:y1, x0:x1] = cv2.addWeighted(card, 1.0 - HUD_ALPHA, np.zeros_like(card), HUD_ALPHA, 0)

```

Two details worth noticing:

- `frame[y0:y1, x0:x1]` is a **view**, not a copy. Writing back into it modifies the original frame in place, which is why the assignment on the second line is necessary rather than optional

— `addWeighted` returns a new array.

- The ROI is clamped to the image, and the function returns early if the clamp produces an empty region: `if x1 <= x0 or y1 <= y0: return frame`. Without that guard, a frame smaller than the card raises an error on the slice assignment. A test covers this case with a 20×20 frame.

Edge-Case Trap 4.4 • Fixed-size cards overflow with two hands

The card is sized dynamically — both dimensions — from the text it must hold:

```

1 sizes = [cv2.getTextSize(line, ...)[0] for line in lines]
2 text_width = max(width for width, _ in sizes)
3 line_height = text_height + 12
4 x1 = min(width, x0 + text_width + 2 * HUD_PAD)
5 y1 = min(height, y0 + 2 * HUD_PAD + line_height * len(lines))

```

A hard-coded box would work until the moment two hands are detected, at which point the HUD gains two extra lines and the text spills out of the card — over the live video, unreadable. Measuring the text first makes the card correct for one hand, two hands, or a long “Rock On” gesture label.

4.6 Teardown and resource safety

```

1 try:
2     while True:
3         ok, frame = capture.read()
4         ...
5 finally:
6     capture.release()
7     tracker.close()
8     cv2.destroyAllWindows()

```

The `finally` block runs on every path out of the loop: the user pressing `q`, an `Esc`, a failed frame read that breaks the loop, or an exception. Each call releases a distinct resource:

- `capture.release()` returns the camera device to the operating system. Skipping it is what produces the “camera already in use” error on the next run.
- `tracker.close()` releases the MediaPipe graph and its native resources.
- `cv2.destroyAllWindows()` removes the GUI window. Without it, a crashed run can leave an orphaned window that keeps a stale frame on screen and is not obviously connected to any process.

The general principle: acquire resources in `\_\_init\_\_` or at the start of a function, and release them in `finally` — never at the end of the happy path only. The happy path is the one case that is already working.

4.7 The full data flow

Chapter summary. The codebase is split along exactly one seam — pure logic versus everything that touches hardware — and that seam is what makes the project testable at all. Each frame converts its landmarks once into a per-hand dictionary, which every downstream rule reads. Opening a camera searches both backends and indices before

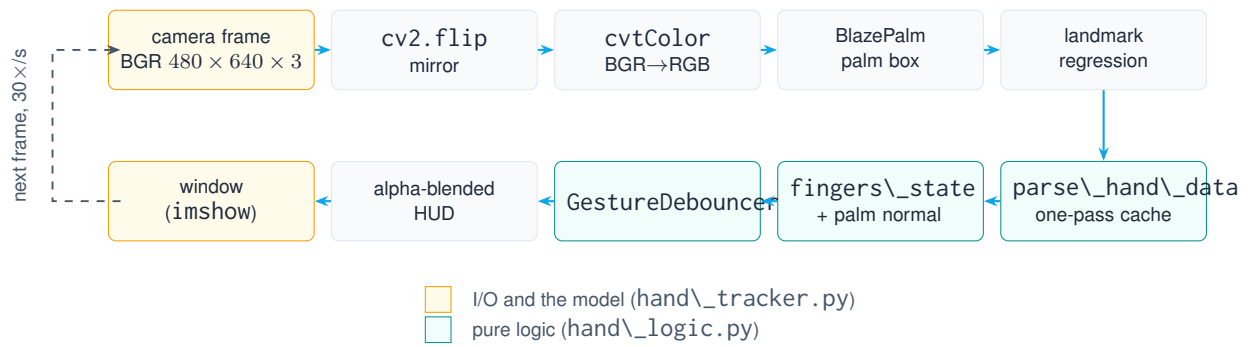


Figure 4.3. One frame, end to end. The trace runs left to right along the top row — mirror, colour conversion, detect, regress — then drops into the pure-logic row and travels back right to left to the window. The dashed line on the left is the only thing making this a video rather than a snapshot.

failing with a message that names the likely cause, and all resources are released in a finally block. The `-image` mode runs the identical pipeline on a still, which is what allows the whole program, drawing included, to be verified in CI without a display server.

Complete Walkthrough of the Source Code

This chapter walks the three source files from top to bottom. Every code block below is **quoted directly out of the project files with L^AT_EX’s `lstinputlisting`**, so the line numbers printed in the margin are the file’s real line numbers — open `hand_logic.py`, jump to line 127, and you will be looking at exactly what is printed here. The complete files are reproduced verbatim in Appendix A.

The three files are presented in dependency order: the pure logic first, then the application that drives it, then the tests that pin both down.

5.1 `hand\_logic.py` — the pure layer

5.1.1 Module docstring, imports and constants

```

1  """Pure hand geometry, landmark parsing and gesture classification.
2
3  This module deliberately imports nothing but the standard library (`math`,
4  `collections`). That keeps every rule below unit-testable with no camera, no
5  OpenCV and no MediaPipe, and lets CI run the logic suite in seconds.
6
7  All pixel coordinates are `(x, y)` with the origin at the top-left of the
8  image, so `y` grows downward: a smaller `y` means "higher on screen".
9  """
10
11 from __future__ import annotations
12
13 import math
14 from collections import deque
15
16 WRIST = 0
17 INDEX_MCP = 5
18 PINKY_MCP = 17
19 THUMB_IP = 3
20 THUMB_TIP = 4
21 INDEX_TIP = 8
22
23 FINGER_PAIRS = [(8, 6), (12, 10), (16, 14), (20, 18)]
24
25 EXPECTED_LANDMARKS = 21
26
27 THUMB_DISTANCE = "distance"
28 THUMB_X = "x"
29
30 NO_GESTURE = ""

```

- **Lines 1–9, the docstring.** It states two things a reader cannot infer from the code: the *dependency policy* (“imports nothing but the standard library”) and the *coordinate convention* (origin top-left, *y* grows downward). The second is the single most consequential fact in the module — every comparison below depends on it — so it is declared before any code.
- **Line 11, `from __future__ import annotations`.** Makes type annotations lazy: they are stored as strings rather than evaluated. That matters because it lets a hint like `list[dict]`

be written on Python 3.8 as well as 3.9+, without a runtime cost. It is the standard first line of any modern Python module.

- **Lines 13–14, the only two imports in the module.** `math` for `math.hypot`, and `deque` for the debouncer’s sliding window. No `cv2`, no `mediapipe` — the dependency arrow from Chapter 4 enforced by construction.
- **Lines 16–21, the landmark index constants.** These are *named* indices. Writing `PINKY\MCP` rather than 17 at the point of use is what allows Section 3.3 to be checked against Section 5.1 line by line: the code and the mathematics use the same names.
- **Line 23, `FINGER\PAIRS`.** A list of (tip, PIP) tuples in the order the state list will use: index, middle, ring, pinky. Note that it deliberately *excludes* the thumb, which follows a completely different rule.
- **Lines 25–30, the remaining constants.** `EXPECTED\_LANDMARKS = 21` is used as a validity check. `THUMB\_DISTANCE` and `THUMB\_X` are the string names of the two thumb rules — string constants rather than booleans, so they read well in a CLI and in an error message. `NO\_GESTURE` is the placeholder emitted when a pose matches no pattern, and — importantly — the initial value of the debouncer, which is what defines its warm-up behaviour.

5.1.2 The distance helper

```

33
34 def _distance(a, b):
35     return math.hypot(a[0] - b[0], a[1] - b[1])

```

- `math.hypot(dx, dy)` computes $\sqrt{dx^2 + dy^2}$ without forming the squares explicitly, and without the overflow that naive squaring can cause for very large values. Here the coordinates are pixels, so overflow is not a concern — but the substitution also avoids a temporary and reads exactly like the formula in Chapter 3.
- The leading underscore is a Python convention: `\_distance` is *internal*. It is not imported by any test, signalling that callers outside the module should not depend on it.

5.1.3 Normalized to pixel conversion, and the bounding box

```

37
38 def to_pixel_landmarks(landmarks, width, height):
39     """Convert normalized landmarks to integer pixel coordinates."""
40     return [(int(lm.x * width), int(lm.y * height)) for lm in landmarks]
41
42
43 def bbox_from_pixels(pixel_landmarks):
44     """Bounding box ``(xmin, ymin, xmax, ymax)`` of a pixel landmark list."""
45     xs = [point[0] for point in pixel_landmarks]
46     ys = [point[1] for point in pixel_landmarks]
47     return (min(xs), min(ys), max(xs), max(ys))

```

- **Line 39** is the comprehension from Chapter 1: for each normalized landmark, multiply *x* by the frame width and *y* by the frame height, and truncate to an integer with `int()`. Note the argument order (*x*, *y*) in the output tuple — pixels are stored as (*x*, *y*) pairs even though the NumPy array is indexed [*y*, *x*]. This module therefore speaks “(*x*, *y*)” consistently, and only the array indexing code in the application has to remember the difference.

- **Line 44–45** build two parallel lists of x and y coordinates. This is the standard idiom for reducing a point list to axis extents, and it is slightly clearer than a running min/max loop.
- **Line 46** returns the box in the conventional order ($x_{\min}, y_{\min}, x_{\max}, y_{\max}$). The order is fixed and documented, because a rectangle whose coordinates are swapped is a silent, plausible-looking bug.

5.1.4 The palm normal

```

49
50 def palm_normal_z(pixel_landmarks):
51     """Z component of the palm-plane normal, from wrist/index-MCP/pinky-MCP.
52
53     ``Z = (x5 - x0)(y17 - y0) - (y5 - y0)(x17 - x0)``
54
55     The sign says which side of the hand faces the camera, which is a useful
56     orientation diagnostic. The convention assumes a mirrored (selfie) view:
57     ``Z < 0`` means the palm faces the camera, ``Z > 0`` the back of the hand.
58     """
59     x0, y0 = pixel_landmarks[WRIST][0], pixel_landmarks[WRIST][1]
60     x5, y5 = pixel_landmarks[INDEX_MCP][0], pixel_landmarks[INDEX_MCP][1]
61     x17, y17 = pixel_landmarks[PINKY_MCP][0], pixel_landmarks[PINKY_MCP][1]
62     return float((x5 - x0) * (y17 - y0) - (y5 - y0) * (x17 - x0))

```

- **Line 50–56, the docstring.** It writes the formula out in the comment as well as the code *and* states the sign convention. For a function whose output is a bare float whose meaning is entirely in its sign, that is essential: without the docstring, the returned `-8775.0` is meaningless.
- **Lines 57–59** destructure the three landmarks used — wrist, index MCP, pinky MCP — into named local variables. This costs one line each and removes every possibility of confusing `x5` with `x17` while reading the formula.
- **Line 60** is the cross product's z -component exactly as derived in Section 3.3. The parenthesis is explicit rather than relying on operator precedence, because a subtle precedence error here would produce a plausible number with the wrong sign.
- `float(...)` on the return is defensive: heights and widths may arrive as NumPy integers, and `np.int64 * np.int64` can overflow at 32 bits. Converting to Python's arbitrary-precision float sidesteps that entirely.

```

64
65 def orientation_from_palm_z(palm_z):
66     """Map the palm-normal sign to the HUD label ``"Palm"`` or ``"Back"``."""
67     return "Palm" if palm_z < 0 else "Back"

```

A three-line function that exists to give a sign a name. It is separate from `palm_normal_z` so that the *convention* lives in exactly one place: if the camera were ever un-mirrored, this single line is what would change.

5.1.5 Reading the handedness label safely

```

69
70 def _handedness_label(handedness, swap_handedness):
71     try:
72         label = handedness.classification[0].label
73     except (AttributeError, IndexError, TypeError):
74         return "Unknown"
75     if label not in ("Left", "Right"):
76         return "Unknown"
77     if swap_handedness:
78         return "Left" if label == "Right" else "Right"
79     return label

```

- **Lines 70–73:** the attribute chain `handedness.classification[0].label` can fail in three distinct ways — the object may lack the attribute, the list may be empty, or the value may not be subscriptable at all. Catching all three exception types in one clause is correct here because the recovery is identical for each: the label is unknown.
- **Lines 74–75:** an unexpected string is normalised to "Unknown" rather than passed through. Everything downstream then has exactly three possible labels, which is what makes the `in ("Left", "Right")` checks elsewhere total.
- **Lines 76–77:** `-swap-handedness` is applied *here*, at parse time, so that the swapped label is what the rest of the program sees — including the state keys used for smoothing and debouncing. Swapping later (at draw time) would have left the state keyed by the original, inconsistent label.

5.1.6 Parsing a frame: the single-pass cache

```

81
82 def parse_hand_data(results, width, height, swap_handedness=False):
83     """Extract everything we need from a MediaPipe result object, once.
84
85     Duck-typed on purpose: it only ever touches ``multi_hand_landmarks`` and
86     ``multi_handedness``, so a fake result object can drive it in tests.
87
88     Returns one dict per detected hand::
89
90         {"label": "Left"|"Right"|"Unknown", "norm": [(x, y, z), ...],
91          "px": [(x, y), ...], "bbox": (xmin, ymin, xmax, ymax),
92          "palm_z": float, "orientation": "Palm"|"Back"}
93
94     ``None``, empty and missing fields all yield ``[]`` this never raises.
95     Hands without a full 21-landmark set are skipped, since no rule below can
96     be evaluated for them.
97     """
98     hand_landmarks = getattr(results, "multi_hand_landmarks", None)
99     if not hand_landmarks:
100         return []
101
102     handedness = getattr(results, "multi_handedness", None) or []
103
104     hands = []
105     for index, hand in enumerate(hand_landmarks):
106         landmarks = getattr(hand, "landmark", None)
107         if not landmarks or len(landmarks) < EXPECTED_LANDMARKS:
108             continue

```

```

109
110     norm = [(lm.x, lm.y, lm.z) for lm in landmarks]
111     px = to_pixel_landmarks(landmarks, width, height)
112     z = palm_normal_z(px)
113     label = _handedness_label(handedness[index], swap_handedness) if index < len(handedness
114 ) else "Unknown"
115
116     hands.append(
117         {
118             "label": label,
119             "norm": norm,
120             "px": px,
121             "bbox": bbox_from_pixels(px),
122             "palm_z": z,
123             "orientation": orientation_from_palm_z(z),
124         }
125     )
126
127     return hands

```

This is the most important function in the module, so it is worth reading line by line.

- **Lines 82–92, the docstring.** It states the duck-typing contract explicitly (“only ever touches multi_hand_landmarks and multi_handedness”), which is the reason the tests can pass a fake object and get real coverage. It also documents the exact dictionary schema — six keys — and the two invariants: never raises, and skips incomplete hands.
- **Lines 93–95,** `getattr(results, "multi\_hand\_landmarks", None)` and the truthiness check. A result with no hands may or may not have the attribute; either way, `None` or an empty list returns `[]`. This one `if` is what makes the “zero hands” and “None input” tests pass without a special case.
- **Lines 97–99:** `or []` converts a present-but-None attribute into an empty list, so the length check on the next line is always safe.
- **Lines 101–104, the loop.** `enumerate` supplies both the index (used to look up the matching handedness entry) and the hand itself. The index correspondence matters: MediaPipe emits the two lists in the same order, which is the only reason `handedness[index]` is correct.
- **Lines 105–107, the completeness gate.** A hand without a full set of 21 landmarks is skipped with `continue` rather than `break`: the *other* hand in the frame may be perfectly usable, and discarding it would be a worse outcome than ignoring the malformed one.
- **Lines 109–110:** `norm` keeps the (x, y, z) triples, and `px` converts with the same helper described above. Both are stored because they serve different consumers — `norm` for debugging and future 3D work, `px` for every rule in the module.
- **Lines 111–124, the dictionary literal.** Every derived value is computed *here and once*: the palm normal, the orientation label, and the bounding box. This is the single-pass cache of Chapter 4 — the reason the render loop performs no redundant coordinate arithmetic.

5.1.7 The finger rules

```

27
28 def fingers_state(pixel_landmarks, hand_label="Unknown", palm_z=None, thumb_mode=THUMB_DISTANCE
29 ):
30     """Return ``[thumb, index, middle, ring, pinky]`` extended flags.
31
32     Indexpinky use the vertical rule from the spec: the fingertip is extended

```

```

32     when it sits strictly above its PIP joint (`y_tip < y_pip`).
33
34     The thumb default is the distance rule, which is robust to handedness and to
35     rotation in the image plane: the tip is extended when it moves away from
36     the pinky knuckle relative to the IP joint. ``thumb_mode=""`` keeps the
37     spec's original horizontal rule (`Right` -> `x_tip < x_ip`, `Left` ->
38     `x_tip > x_ip`) available for reference; it is only valid for an upright,
39     palm-facing hand.
40
41     ``palm_z`` is accepted for API symmetry with the orientation readout; the
42     default distance rule does not need it (it is already handedness-agnostic).
43
44     A ``None``, malformed or short landmark list returns exactly ``[False] * 5``
45     and never raises.
46     """
47     if not pixel_landmarks or len(pixel_landmarks) < EXPECTED_LANDMARKS:
48         return [False] * 5
49
50     try:
51         states = [pixel_landmarks[tip][1] < pixel_landmarks[pip][1] for tip, pip in
52                   FINGER_PAIRS]
53
54         if thumb_mode == THUMB_X and hand_label in ("Left", "Right"):
55             tip, ip = pixel_landmarks[THUMB_TIP], pixel_landmarks[THUMB_IP]
56             thumb = tip[0] < ip[0] if hand_label == "Right" else tip[0] > ip[0]
57         else:
58             thumb = _distance(pixel_landmarks[THUMB_TIP], pixel_landmarks[PINKY_MCP]) >
59                   _distance(
60                       pixel_landmarks[THUMB_IP], pixel_landmarks[PINKY_MCP]
61                   )
62     except (IndexError, TypeError, ValueError):
63         return [False] * 5
64
65     return [thumb] + states

```

- **Lines 128–139, the docstring.** It documents all three rules — the vertical test for the four long fingers, the distance rule for the thumb, and the alternative horizontal rule — plus the exact safe-result contract. It also explains why `palm_z` is accepted but unused by the default rule (API symmetry with the orientation readout), which prevents a future reader from assuming the parameter is a bug.
- **Lines 141–142, the input guard.** This single line implements the specification's requirement: a `None`, empty, or short landmark list returns `[False] * 5` and never raises. Note it is a *list comprehension-free* early return, evaluated before any indexing can occur.
- **Line 146, the four-finger test.** A single comprehension over `FINGER_PAIRS`, comparing the *y* of each tip against the *y* of its PIP joint — the rule derived in Section 3.1. The result is a list of exactly four booleans, in the order the pairs are declared.
- **Lines 148–150, the horizontal thumb rule.** Two conditions guard it: an explicit `thumb_mode == THUMB_X` and a usable label. If either fails, control falls to the `else` branch — so an "Unknown" label degrades to the distance rule instead of silently reporting a folded thumb. That fallback is asserted by a test.
- **Lines 151–154, the distance rule.** The default path, and the return of Section 3.2's derivation. Written as a direct transcription of the inequality, which is why it is readable without a comment.
- **Lines 155–158, the exception guard.** The commented contract is “never raises”, and a comprehension over user-supplied data cannot guarantee that. Catching `IndexError`,

`TypeError` and `ValueError` covers short lists, `None` elements and non-numeric entries, and all three recover to the same safe value. A test feeds a list of 21 `None` values specifically to exercise this path.

- **Line 160:** the thumb result is *prepended* to the four finger results, giving the canonical [thumb, index, middle, ring, pinky] order that every consumer — classification, HUD text, and the tests — relies on.

5.1.8 Gesture classification

```

65
66 def classify_gesture(states):
67     """Name the pose from a 5-element finger-state list."""
68     if not states or len(states) < 5:
69         return NO_GESTURE
70
71     thumb, index, middle, ring, pinky = (bool(state) for state in states[:5])
72
73     if thumb and index and middle and ring and pinky:
74         return "Open Palm"
75     if not (thumb or index or middle or ring or pinky):
76         return "Fist"
77     if thumb and not (index or middle or ring or pinky):
78         return "Thumbs Up"
79     if not thumb and index and middle and not (ring or pinky):
80         return "Peace"
81     if thumb and index and not (middle or ring) and pinky:
82         return "Rock On"
83     return NO_GESTURE

```

- **Lines 166–167:** a short or empty state list returns the placeholder rather than raising, keeping the module’s no-exceptions contract intact.
- **Line 169** unpacks exactly five values and coerces each with `bool()`. The coercion matters: it means a caller passing `1/0`, or NumPy booleans, gets the same behaviour as one passing `True/False`.
- **Lines 171–180, the pattern chain.** Each test is *exhaustive* — it states which fingers must be down as well as which must be up. Without the negated terms, “Peace” would also match an open palm, and the first match would win. The ordering from most to least obvious is a readability choice, not a correctness requirement, because the patterns are mutually exclusive.
- **Line 182:** anything unmatched returns `NO_GESTURE`. The module never returns `None` for a gesture, so the HUD can always print something.

5.1.9 The debouncer

```

85
86 class GestureDebouncer:
87     """Rolling majority vote that suppresses single-frame gesture flicker.
88
89     ``update`` returns the new label once at least ``min_agree`` of the last
90     ``window`` frames agree on it, and otherwise keeps returning the last stable
91     label. Before any majority has ever formed it returns ``""``.
92     """
93

```

```

194     def __init__(self, window=5, min_agree=4):
195         self.window = window
196         self.min_agree = min_agree
197         self._history = deque(maxlen=window)
198         self._stable = NO_GESTURE
199
200     @property
201     def stable(self):
202         return self._stable
203
204     def update(self, gesture):
205         self._history.append(gesture)
206         if self._history.count(gesture) >= self.min_agree:
207             self._stable = gesture
208         return self._stable

```

- **Lines 185–191, the class docstring.** It states the contract in three clauses: the window length, the agreement threshold, and — crucially — the warm-up behaviour (“before any majority has ever formed it returns –”). That last sentence is the answer to a question the code alone leaves ambiguous.
- **Lines 193–198, the constructor.** Three pieces of state per instance: `window` and `min_agree` as configuration, `_history` as a `deque(maxlen=window)` that evicts automatically, and `_stable` seeded with the placeholder. Seeding with the placeholder *is* the warm-up policy.
- **Lines 200–201, the `stable` property.** A read-only view of the last accepted label. Exposing it via `@property` rather than a plain attribute means callers cannot accidentally assign to it — a small invariant guard.
- **Lines 203–207, `update`.** Three statements: append the new observation (the deque silently evicts the oldest), test whether the *new* label alone has reached the threshold within the window, and return the current stable label either way. Counting only the newest label is correct, and cheaper than building a histogram of all five entries: if any label reached the threshold, the newest one is the only candidate that could have just crossed it.

5.1.10 The exponential moving average

```

210
211     def ema(previous, current, alpha=0.65):
212         """Exponential moving average: ``alpha * current + (1 - alpha) * previous``.
213
214         ``previous is None`` (first sighting) returns ``current`` unchanged.
215         """
216         if previous is None:
217             return (current[0], current[1])
218         return (
219             alpha * current[0] + (1 - alpha) * previous[0],
220             alpha * current[1] + (1 - alpha) * previous[1],
221         )

```

- **Lines 211–214, the docstring,** which writes the recurrence in words and states the first-sample rule.
- **Lines 215–216:** the `None` special case. Without it, the arithmetic would raise, or — worse, if a caller substituted zero — the dot would animate in from the origin on the first frame.

- **Lines 217–220:** the weighted sum, applied to x and y independently but in one expression, returning a plain tuple. Floats are returned, not integers: rounding here would accumulate error and defeat the purpose of smoothing. The caller rounds once, at draw time.

5.2 hand_tracker.py — the application

5.2.1 Module setup, the import guard, and constants

```

36 LOG = logging.getLogger("hand_tracker")
37
38 with warnings.catch_warnings():
39     warnings.simplefilter("ignore")
40     import mediapipe as mp
41
42 if not hasattr(mp, "solutions") or not hasattr(mp.solutions, "hands"):
43     raise RuntimeError(
44         "mediapipe>=0.10.30 removed mp.solutions, which this app is built on. "
45         "Install the pinned version instead: pip install -r requirements.txt "
46         "(do NOT `pip install -U mediapipe`)."
47     )
48
49 with warnings.catch_warnings():
50     warnings.simplefilter("ignore")
51     mp_hands = mp.solutions.hands
52     mp_drawing = mp.solutions.drawing_utils
53
54 WINDOW_NAME = "Hand Tracker"
55 CAPTURES_DIR = "captures"
56
57 DEFAULT_WIDTH = 640
58 DEFAULT_HEIGHT = 480
59
60 HUD_FONT = cv2.FONT_HERSHEY_SIMPLEX
61 HUD_FONT_SCALE = 0.6
62 HUD_FONT_THICKNESS = 1
63 HUD_MARGIN = 10
64 HUD_PAD = 10
65 HUD_ALPHA = 0.6
66
67 INDEX_TIP_HALO_RADIUS = 14
68 INDEX_TIP_RADIUS = 10
69 INDEX_TIP_COLOR = (0, 165, 255)
70
71 FPS_WINDOW = 30

```

- **Line 36:** the module logger. Every diagnostic in the application goes through it rather than print, so verbosity is controllable from the command line and library users are not spammed by a chatty dependency.
- **The `warnings.catch\_warnings()` block** around the MediaPipe import suppresses the deprecation noise of the legacy API — scoped to the import, so real warnings elsewhere still surface. This is discussed in detail in Section 2.3.
- **The `RuntimeError` guard** is the third dependency trap from Chapter 2: it converts an obscure `AttributeError` from deep inside a library into one actionable sentence naming the fix.
- **The `tikz-free constants block` (lines 54–71)** collects every magic number the application uses: the window title, the captures directory, the default frame size, the six HUD styling values, the two index-tip marker radii and its colour, and the FPS averaging window.

Collecting them means the look of the overlay can be tuned without hunting through the drawing code — and it makes each one citable in this handbook.

- `INDEX\_TIP\_COLOR = (0, 165, 255)` deserves a note: this is **BGR**, not RGB. Read as a colour, it is “no blue, 165 green, 255 red” — the orange dot visible in the running application. A constant written as `(255, 165, 0)` would draw a *blue* marker, and it would look deliberate.

5.2.2 HandTracker: construction and the detection call

```

77     def __init__(
78         self,
79         max_hands=2,
80         mode=False,
81         det_conf=0.7,
82         track_conf=0.6,
83         thumb_mode="distance",
84         swap_handedness=False,
85     ):
86         self.thumb_mode = thumb_mode
87         self.swap_handedness = swap_handedness
88         self.hands_data = []
89         self._smooth_tip = {}
90         self._debounce = {}
91         self.hands = mp_hands.Hands(
92             static_image_mode=mode,
93             max_num_hands=max_hands,
94             min_detection_confidence=det_conf,
95             min_tracking_confidence=track_conf,
96         )
97
98     def find_hands(self, frame, draw=True):
99         """Detect hands in a BGR frame, draw the mesh, and rebuild the cache.
100
101         The normalized landmarks, pixel landmarks, bounding box, handedness
102         label and palm orientation are computed once here and reused by every
103         helper below for the rest of the frame.
104         """
105         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
106         rgb.flags.writeable = False
107         results = self.hands.process(rgb)
108         rgb.flags.writeable = True
109
110         height, width = frame.shape[:2]
111         self.hands_data = parse_hand_data(results, width, height, self.swap_handedness)
112
113         if draw and results.multi_hand_landmarks:
114             for hand_landmarks in results.multi_hand_landmarks:
115                 mp_drawing.draw_landmarks(
116                     frame,
117                     hand_landmarks,
118                     mp_hands.HAND_CONNECTIONS,
119                     mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2, circle_radius=2),
120                     mp_drawing.DrawingSpec(color=(235, 235, 235), thickness=2),
121                 )
122         return frame

```

- **Lines 77–96, the constructor.** The five parameters mirror the CLI flags, each with the default the specification requires. Four of them are configuration; the fifth, `mode`, is passed straight to `static\_image\_mode`. Note the ordering of assignments: the three caches are created before the MediaPipe model, so an object is never in a half-built state even if model

construction raises.

- **Line 90, the model.** This is the only place in the program where MediaPipe is instantiated. Isolating construction to one line makes the three thresholds from Chapter 2 easy to find and tune.
- **Line 91–93, the three caches.** `hands\_data` is the per-frame telemetry; `\_smooth\_tip` holds one EMA state per hand key; `\_debounce` holds one GestureDebounce per hand key. All three are *instance* attributes — two trackers would not share state.
- **Lines 98–114, the detection itself.** The conversion to RGB, the read-only flag, the model call, and the restore of the write flag. Setting `rgb.flags.writeable = False` is a micro-optimisation that tells NumPy the buffer will not be written, letting the model read it without defensive copying.
- **Lines 116–119, the cache rebuild.** Note the order: the image is measured (`frame.shape[:2]` gives *height*, *width*), and `parse\_hand\_data` is called with (*width*, *height*) — the opposite order, because the function's parameters are named *width*, *height*. This is the single most error-prone line in the file, and it is correct precisely because both names are explicit at the call site.
- **Lines 121–122, the optional mesh.** Drawing is guarded by both the draw flag and the presence of landmarks. `mp\_drawing.draw\_landmarks` is handed two styles: a green joint style and a near-white connection style, which is what produces the familiar MediaPipe skeleton.

5.2.3 The accessor methods

```

24 def hand_count(self):
25     """Number of hands seen in the most recent ``find_hands`` call."""
26     return len(self.hands_data)
27
28 def hand_label(self, hand_index):
29     """Handedness label of a detected hand, or ``Unknown``."""
30     if 0 <= hand_index < len(self.hands_data):
31         return self.hands_data[hand_index]["label"]
32     return "Unknown"
33
34 def find_positions(self, frame, hand_index=0):
35     """Cached pixel landmarks for one hand, or ``[]`` if there is none.
36
37     ``frame`` is only kept for signature compatibility: the returned points
38     were computed inside the last ``find_hands`` call, so they are only
39     valid for a frame of that same size.
40     """
41     if 0 <= hand_index < len(self.hands_data):
42         return list(self.hands_data[hand_index]["px"])
43     return []
44
45 def fingers_up(self, landmarks, hand_label):
46     """``[thumb, index, middle, ring, pinky]`` extended flags for one hand."""
47     return fingers_state(landmarks, hand_label, thumb_mode=self.thumb_mode)

```

- `hand\_count` is a one-liner over the cache. It is a method rather than a direct attribute read so that the length and the cache can never disagree.
- `hand\_label` returns "Unknown" out of range — never raises, and never returns None, so its result can be placed directly into HUD text.

- **find_positions** is the specification’s method, preserved exactly. Two things about it are deliberate: it returns a *copy* (`list(...)`) so a caller cannot mutate the cache, and it returns `[]` — not `None` — for a bad index, so the finger rules receive a value they already have a guard for. The docstring records the vestigial frame parameter and the size caveat from Section 4.2.
- **fingers_up** is a one-line delegation to `fingers\_state`, carrying the instance’s `thumb\_mode` so the CLI flag reaches the rule without the render loop having to pass it around.

5.2.4 The smoothed tip, orientation, and debounced gesture

```

49 def index_tip(self, hand_index, alpha=0.65):
50     """EMA-smoothed pixel position of landmark 8, or ``None``."""
51     if not 0 <= hand_index < len(self.hands_data):
52         return None
53
54     hand = self.hands_data[hand_index]
55     if len(hand["px"]) <= INDEX_TIP:
56         return None
57
58     key = self._state_key(hand_index, hand["label"])
59     smoothed = ema(self._smooth_tip.get(key), hand["px"][INDEX_TIP], alpha)
60     self._smooth_tip[key] = smoothed
61     return (int(smoothed[0]), int(smoothed[1]))
62
63 def orientation(self, hand_index):
64     """`"Palm" or "Back" for a detected hand, or ``None``."""
65     if 0 <= hand_index < len(self.hands_data):
66         return self.hands_data[hand_index]["orientation"]
67     return ""
68
69 def gesture(self, hand_index, states):
70     """Debounced gesture name for the finger states of one hand."""
71     key = self._state_key(hand_index, self.hand_label(hand_index))
72     debouncer = self._debounce.get(key)
73     if debouncer is None:
74         debouncer = GestureDebouncer()
75         self._debounce[key] = debouncer
76     return debouncer.update(classify_gesture(states))
77
78 @staticmethod

```

- **Lines 150–152, `index\_tip`’s guards.** Out-of-range index and a too-short landmark list both return `None`. `None` is the right sentinel here — unlike `find\_positions`, the caller genuinely has nothing to draw — and the render loop checks for it before calling `cv2.circle`.
- **Line 157, the state key.** The EMA state is looked up by hand identity, not by list position, which is the fix for the cross-contamination trap in Section 3.4.
- **Lines 158–160:** the smoothing call, the state update, and the cast to integers. Note that the *float* value is what gets stored, and the rounding happens only on the way out — so the filter’s memory is never quantised.
- **`orientation`** returns the cached label, with an em-dash for out-of-range. It performs no arithmetic at all: the sign test was done once, in `parse\_hand\_data`.
- **Lines 169–178, `gesture`.** The debouncer is created lazily on first use for a given key and then reused. Lazy creation is what allows the tracker to adapt to hands appearing and disappearing mid-session without a separate bookkeeping pass — and because the key is the

label, a hand that briefly leaves the frame and returns resumes its own gesture history rather than inheriting another hand's.

```

79
80     def _state_key(hand_index, label):
81         """Key per-hand smoothing/debounce state.
82
83         MediaPipe's hand list order is not stable across frames, so keying by
84         list index would blend two different physical hands together when they
85         reorder the label is a much steadier identity.
86         """
87         return label if label in ("Left", "Right") else f"index:{hand_index}"
88
89     def close(self):
90         self.hands.close()

```

- `\_state\_key` is a static method because it needs no instance state — it is a pure function of its arguments, and marking it `@staticmethod` says so.
- The fallback key, `f"index:\{hand\_index\}"`, is deliberately namespaced with a prefix. If the label list ever contained the literal string `"0"`, the two key spaces could collide; the prefix makes that impossible.
- `close` releases the MediaPipe graph. It exists so the teardown in Section 4.6 can call one method rather than reaching into `self.hands`, keeping the model's lifecycle inside the class that owns it.

5.2.5 The HUD: text assembly and drawing

```

192
193     def hud_lines(total_fingers, hand_lines, fps_text):
194         """Assemble the HUD text: FPS, total count, then one line per hand."""
195         return [f"FPS: {fps_text}", f"Fingers: {total_fingers}"] + list(hand_lines)
196
197
198     def draw_hud(frame, lines):
199         """Draw an alpha-blended card behind white telemetry text (top-left)."""
200         if not lines:
201             return frame
202
203         sizes = [cv2.getTextSize(line, HUD_FONT, HUD_FONT_SCALE, HUD_FONT_THICKNESS)[0] for line in
204                 lines]
205         text_width = max(width for width, _ in sizes)
206         text_height = max(height for _, height in sizes)
207         line_height = text_height + 12
208
209         height, width = frame.shape[:2]
210         x0, y0 = HUD_MARGIN, HUD_MARGIN
211         x1 = min(width, x0 + text_width + 2 * HUD_PAD)
212         y1 = min(height, y0 + 2 * HUD_PAD + line_height * len(lines))
213         if x1 <= x0 or y1 <= y0:
214             return frame
215
216         card = frame[y0:y1, x0:x1]
217         frame[y0:y1, x0:x1] = cv2.addWeighted(card, 1.0 - HUD_ALPHA, np.zeros_like(card), HUD_ALPHA,
218         , 0)
219
220         y = y0 + HUD_PAD + text_height
221         for line in lines:

```

```

220         cv2.putText(
221             frame,
222             line,
223             (x0 + HUD_PAD, y),
224             HUD_FONT,
225             HUD_FONT_SCALE,
226             (255, 255, 255),
227             HUD_FONT_THICKNESS,
228             cv2.LINE_AA,
229         )
230         y += line_height
231     return frame

```

- **Lines 192–194, `hud\_lines`.** A pure function that assembles the text: FPS, total, then one line per hand. Separating assembly from drawing means the text can be tested without a frame — and it is, by `test\_hud\_lines\_formatting`.
- **Lines 198–199:** an empty line list returns immediately, so an empty card is never drawn.
- **Lines 201–205, `measurement`.** `cv2.getTextSize` returns the pixel extent of a string in the chosen font. Measuring *before* drawing is what allows the card to size itself to its content, which is the fix for the two-hand overflow trap in Section 4.5.
- **Lines 207–214, the clamped rectangle.** Four values are computed: the top-left corner, and the bottom-right corner clamped to the image with `min`. The early return on `x1 <= x0` or `y1 <= y0` handles a frame so small that the card would be degenerate — 20-pixel-square frames do occur in tests and in thumbnail previews.
- **Lines 216–217, the blend.** The ROI is a NumPy *view*; `addWeighted` reads it, and the assignment writes the darkened result back into the same memory. The expression `1.0 - HUD\_ALPHA` keeps the two weights summing to 1, so brightness is scaled rather than shifted — which is why the card looks like tinted glass rather than a black box.
- **Lines 219–230, the text loop.** The baseline advances by `line\_height` each iteration, starting one pad plus one text height below the card’s top edge. White (255, 255, 255) is chosen because the blended background is guaranteed dark; the `cv2.LINE\_AA` flag enables anti-aliasing so small text stays legible.

5.2.6 The per-frame loop body

```

233
234 def annotate_hands(tracker, frame):
235     """Run the finger logic and overlays for every detected hand.
236
237     Returns the total number of extended fingers and one HUD line per hand.
238     """
239     total = 0
240     lines = []
241
242     for index in range(tracker.hand_count()):
243         label = tracker.hand_label(index)
244         landmarks = tracker.find_positions(frame, index)
245         states = tracker.fingers_up(landmarks, label)
246         count = sum(states)
247         total += count
248         lines.append(f"{label}: {count} ({tracker.gesture(index, states)}) [{tracker.
249             orientation(index)}]")

```

```

250     tip = tracker.index_tip(index)
251     if tip is not None:
252         cv2.circle(frame, tip, INDEX_TIP_HALO_RADIUS, (255, 255, 255), cv2.FILLED)
253         cv2.circle(frame, tip, INDEX_TIP_RADIUS, INDEX_TIP_COLOR, cv2.FILLED)
254
255     return total, lines

```

This function is the shared heart of both the live loop and `-image` mode — which is what guarantees the still-image path exercises the real drawing code rather than a copy of it.

- **Line 238, the loop bound.** `range(tracker.hand\_count())` makes both hands in the frame fall out naturally, and makes the two-hand requirement of the specification a non-event.
- **Lines 239–242, the read-and-read-again pattern.** `hand\_label` and `find\_positions` both come from the cache populated by the preceding `find\_hands`. No landmark is recomputed below this line.
- **Lines 243–244:** `sum(states)` counts the True values directly. Since the booleans are Python `bool`, which is a subclass of `int`, this is both correct and idiomatic.
- **Line 245, the HUD row.** An f-string with four fields: label, count, debounced gesture, and the palm orientation in square brackets. This single line is the “diagnostic visual telemetry” the specification asks for, and every field is traceable to a rule derived in Chapter 3.
- **Lines 247–251, the marker.** Two filled circles: a white halo of radius 14, then the orange dot of radius 10 on top. Drawing the halo first and the dot second is what produces a ringed marker that stays visible against both light and dark backgrounds — the same design principle as the HUD card.

5.2.7 Screenshots, and the camera fallback chain

```

257
258 def save_screenshot(frame):
259     """Write the annotated frame (HUD burned in) to ``captures/``."""
260     os.makedirs(CAPTURES_DIR, exist_ok=True)
261     stamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")[:-3]
262     path = os.path.join(CAPTURES_DIR, f"frame_{stamp}.png")
263     if cv2.imwrite(path, frame):
264         LOG.info("Saved screenshot: %s", path)
265     else:
266         LOG.error("Could not write screenshot: %s", path)
267
268
269 def open_camera(index, width, height):
270     """Open a webcam, falling back across backends and camera indices."""
271     backends = [cv2.CAP_DSHOW, cv2.CAP_MSMF, cv2.CAP_ANY] if os.name == "nt" else [cv2.CAP_ANY]
272
273     for candidate in [index] + [other for other in (1, 0) if other != index]:
274         for backend in backends:
275             capture = cv2.VideoCapture(candidate, backend)
276             if capture.isOpened():
277                 capture.set(cv2.CAP_PROP_FRAME_WIDTH, width)
278                 capture.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
279                 LOG.info(
280                     "Camera %d opened at %dx%d.",
281                     candidate,
282                     int(capture.get(cv2.CAP_PROP_FRAME_WIDTH)),
283                     int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT)),
284                 )

```

```

285         return capture
286         capture.release()
287         LOG.warning("Camera %d did not open.", candidate)
288
289     LOG.error("Could not open a webcam. Check it is connected and not in use by another app.")
290     return None

```

- **save_screenshot** (lines 257–265) creates the directory if needed — the `exist\_ok=True` flag makes repeated presses idempotent rather than errors — and builds a timestamped filename. The timestamp uses `%f` microseconds truncated to milliseconds (`[:-3]`), so two presses within the same second cannot overwrite each other.
- The `if cv2.imwrite(...)` test is not paranoia: `imwrite` returns `False` on failure (a full disk, a permission problem, an unsupported extension) rather than raising, so an unchecked call would fail silently and the user would simply never find the file.
- **open_camera** (lines 268–289) is the fallback chain of Section 4.3, covered in detail there. Two details are worth re-reading in the code: the backend list is chosen by `os.name == "nt"`, and the candidate list is built by filtering the requested index out of `(1, 0)` so no index is ever tried twice.

5.2.8 The live capture loop

```

292
293 def run_live(args):
294     """Webcam capture loop: read, flip, analyse, draw, show."""
295     capture = open_camera(args.camera, args.width or DEFAULT_WIDTH, args.height or
296                           DEFAULT_HEIGHT)
297     if capture is None:
298         return 1
299
300     tracker = HandTracker(
301         max_hands=args.max_hands,
302         mode=args.static,
303         thumb_mode=args.thumb_mode,
304         swap_handedness=args.swap_handedness,
305     )
306     frame_times = deque(maxlen=FPS_WINDOW)
307     previous_time = time.time()
308
309     try:
310         while True:
311             ok, frame = capture.read()
312             if not ok:
313                 LOG.error("Frame capture failed; stopping.")
314                 break
315
316             frame = cv2.flip(frame, 1)
317             frame = tracker.find_hands(frame, draw=not args.no_draw)
318             total, hand_lines = annotate_hands(tracker, frame)
319
320             now = time.time()
321             delta = now - previous_time
322             previous_time = now
323             if delta > 0:
324                 frame_times.append(1.0 / delta)
325                 fps_text = f"{sum(frame_times) / len(frame_times):.0f}" if frame_times else "0"
326
327             draw_hud(frame, hud_lines(total, hand_lines, fps_text))

```

```

327         cv2.imshow(WINDOW_NAME, frame)
328
329         key = cv2.waitKey(1) & 0xFF
330         if key in (ord("q"), 27):
331             break
332         if key == ord("s"):
333             save_screenshot(frame)
334     finally:
335         capture.release()
336         tracker.close()
337         cv2.destroyAllWindows()
338
339     return 0

```

- **Lines 294–296:** the camera is opened *first*, and a `None` return exits with status 1 before any model is constructed. Failing fast here means a missing webcam never pays the several-hundred-millisecond cost of loading the MediaPipe graph.
- **Lines 304–305:** a `deque(maxlen=30)` of instantaneous frame rates and a timestamp. The window gives a one-second moving average at 30 FPS — long enough to smooth the reading, short enough to react to a real slowdown.
- **Lines 314–316, the per-frame core.** Flip, detect, annotate. Three lines, and `annotate\_hands` is shared with `-image` mode.
- **Lines 318–323, the FPS calculation.** This is the `time.time()` delta approach the specification requires: measure the interval since the previous frame, invert it for an instantaneous rate, and average the last thirty. The `if delta > 0` guard protects against a clock with insufficient resolution returning the same timestamp twice — which would otherwise divide by zero.
- **Lines 325–326:** the HUD is drawn and the frame is shown. The order matters — `imshow` displays whatever is in the buffer at that moment, so drawing after it would show a frame-delayed overlay.
- **Lines 328–332, input handling.** `cv2.waitKey(1)` waits one millisecond for a keypress, which also pumps the GUI event loop — without it the window would freeze. `& 0xFF` masks off platform-specific bits above the ASCII range, so the comparison against `ord("q")` works on every platform. Key 27 is Esc.
- **Lines 333–336, the finally block:** camera released, model closed, windows destroyed — on every exit path, as required by Section 4.6.

5.2.9 Headless image mode

```

341
342 def run_image(args):
343     """Run the full pipeline once over a still image and write ``<stem>_out.png``."""
344     frame = cv2.imread(args.image)
345     if frame is None:
346         LOG.error("Could not read image: %s", args.image)
347         return 1
348
349     if args.width and args.height:
350         frame = cv2.resize(frame, (args.width, args.height))
351         LOG.info("Resized input to %dx%d.", args.width, args.height)
352
353     tracker = HandTracker(

```

```

354         max_hands=args.max_hands,
355         mode=True,
356         thumb_mode=args.thumb_mode,
357         swap_handedness=args.swap_handedness,
358     )
359     try:
360         frame = tracker.find_hands(frame, draw=not args.no_draw)
361         total, hand_lines = annotate_hands(tracker, frame)
362         draw_hud(frame, hud_lines(total, hand_lines, "n/a (still)")
363         hands_seen = tracker.hand_count()
364     finally:
365         tracker.close()
366
367     stem, _ = os.path.splitext(args.image)
368     output_path = f"{stem}_out.png"
369     if not cv2.imwrite(output_path, frame):
370         LOG.error("Could not write output image: %s", output_path)
371         return 1
372
373     LOG.info("Detected %d hand(s) with %d extended finger(s); wrote %s", hands_seen, total,
374             output_path)
375     return 0

```

- **Lines 342–346:** `cv2.imread` returns `None` for a missing or unreadable file — it does not raise. Checking for `None` and returning 1 is what makes `-image does-not-exist.png` a clean failure that a CI script can detect from the exit code.
- **Lines 348–350:** resizing happens only when *both* dimensions were supplied. The default of “native resolution” is the right choice for stills, where there is no frame-rate argument for downscaling.
- **Lines 352–357:** the tracker is constructed with `mode=True` (`static\_image\_mode`), because a single frame has no successor to track into.
- **Lines 359–363, the shared pipeline.** `find\_hands` → `annotate\_hands` → `draw\_hud` — the identical functions the live loop uses. The `try/finally` closes the model even if detection raises.
- **Lines 365–373, the output path.** `os.path.splitext` splits the input into stem and extension; the stem plus `\_out.png` is written *next to the input*, so running the mode twice is idempotent and no directory configuration is needed. The reported `hand\_count()` comes from the cache built during `find\_hands`, which is why the log line can state a hand count without re-running detection.

5.2.10 The command-line interface and entry point

```

376
377 def build_parser():
378     parser = argparse.ArgumentParser(
379         prog="hand_tracker.py",
380         description="Real-time hand and finger tracker (MediaPipe + OpenCV).",
381     )
382     parser.add_argument("-camera", type=int, default=0, help="webcam index (default: 0)")
383     parser.add_argument(
384         "-width", type=int, default=None, help="capture width (default: 640 live, native in -
385         image)"
386     )
387     parser.add_argument(

```

```

387     "-height", type=int, default=None, help="capture height (default: 480 live, native in -
388     image)"
389 )
390 parser.add_argument("-max-hands", type=int, default=2, help="maximum hands to detect (
391     default: 2)")
392 parser.add_argument("-no-draw", action="store_true", help="disable the skeletal mesh
393     overlay")
394 parser.add_argument(
395     "-thumb-mode",
396     choices=("distance", "x"),
397     default="distance",
398     help="thumb rule: 'distance' (rotation-robust, default) or 'x' (original horizontal
399     rule)",
400 )
401 parser.add_argument(
402     "-swap-handedness",
403     action="store_true",
404     help="swap Left/Right labels (display-only fix if your mirrored feed reports them
405     inverted)",
406 )
407 parser.add_argument("-static", action="store_true", help="use MediaPipe static_image_mode (
408     no tracking)")
409 parser.add_argument("-image", metavar="PATH", help="process a still image instead of the
410     camera, then exit")
411 parser.add_argument(
412     "-log-level",
413     default="INFO",
414     choices=("DEBUG", "INFO", "WARNING", "ERROR"),
415     help="logging verbosity (default: INFO)",
416 )
417 return parser
418
419 def main(argv=None):
420     args = build_parser().parse_args(argv)
421     logging.basicConfig(level=getattr(logging, args.log_level), format="%(levelname)s: %(
422     message)s")
423
424     if args.image:
425         return run_image(args)
426     return run_live(args)
427
428 if __name__ == "__main__":
429     raise SystemExit(main())

```

- **Lines 377–381:** argparse is configured with a program name and a description, so `-help` produces usable output rather than a bare usage string.
- **Lines 382–384, the camera and size flags.** `-width` and `-height` default to `None`, not to 640 and 480. That deliberate choice is what allows `-image` mode to distinguish “the user asked for a resize” from “the user accepted a default”; the live loop supplies 640×480 itself when the flags are absent.
- **Line 388, `-thumb-mode`** uses `choices=("distance", "x")`. argparse then rejects a typo such as `-thumb-mode distancey` with a clear message, rather than silently falling back to the default.
- **Line 403, `-image`:** the flag that makes the whole application testable in CI, described in Section 4.4.
- **Line 412–419, `main`.** Logging is configured from the parsed level, and the function returns

an integer — 0 or 1 — rather than calling `sys.exit` itself. That makes `main()` callable from a test with an argument list, which is how a CLI is normally tested.

- **Lines 421–422, the entry point.** `raise SystemExit(main())` converts the return value into the process exit code, and the `\_name\_` guard ensures that importing the module (as the test suite does) never starts a camera.

5.3 tests/test_hand_logic.py — the offline test suite

5.3.1 The synthetic hands

The file's first seventy lines define two coordinate sets by hand: `OPEN\_HAND` (all five fingers extended) and `FIST` (all five folded). They are ordinary lists of (x, y) tuples with a comment naming each landmark. Because they are literals rather than captured data, every assertion below is exact — there is no tolerance and no randomness.

Four helpers manipulate them:

```

80 def _with_finger(points, tip_index, tip_position):
81     updated = list(points)
82     updated[tip_index] = tip_position
83     return updated
84
85
86 def _with_thumb(points, cmc, mcp, ip, tip):
87     updated = list(points)
88     updated[1], updated[2], updated[3], updated[4] = cmc, mcp, ip, tip
89     return updated
90
91
92 def _rotate(points, degrees):
93     """Rotate every landmark around the set's centroid."""
94     radians = math.radians(degrees)
95     cos_a, sin_a = math.cos(radians), math.sin(radians)
96     cx = sum(p[0] for p in points) / len(points)
97     cy = sum(p[1] for p in points) / len(points)
98     return [
99         (
100             round(cx + cos_a * (x - cx) - sin_a * (y - cy)),
101             round(cy + sin_a * (x - cx) + cos_a * (y - cy)),
102         )
103         for x, y in points
104     ]
105
106
107 def _mirror_x(points):
108     return [(-x, y) for x, y in points]
109
110
111 # -----
112 # Finger state
113 # -----

```

- `\_with\_finger` and `\_with\_thumb` (lines 80–89) return a modified *copy* of a hand with one finger, or the whole thumb, moved to a chosen position. Copying is essential: mutating the shared literal would make the tests order-dependent, so a test that passed alone could fail when run after another.
- `\_rotate` (lines 92–104) applies a real rotation matrix about the point set's centroid, rounding

the result back to integers. It is the machinery behind the project’s most important regression test — the one that encodes the reviewer’s original complaint about the horizontal thumb rule.

- `\_mirror\_x` (lines 107–113) negates every x coordinate. Geometrically that is “show the other side of the hand”, and it is exactly the transformation that must flip the palm normal’s sign.

5.3.2 The rotation-invariance test — the project’s key regression

```

73 def test_thumb_survives_image_plane_rotation():
74     """The regression test for the old x-only rule: rotate the hand 90 degrees."""
75     for degrees in (90, 37, 180, -120):
76         rotated = _rotate(OPEN_HAND, degrees)
77         assert fingers_state(rotated)[0] is True, f"thumb lost at {degrees} degrees"

```

This four-line test is the reason the default thumb rule exists. It rotates the same open hand through four angles and asserts that the thumb state never changes. The old horizontal rule fails this test at 90° and 180°; the distance rule passes at every angle, for the algebraic reason proved in Section 3.2. Turning a reviewer’s complaint into an executable assertion is what stops the bug from ever returning silently.

5.3.3 The safe-result contract

```

86 def test_malformed_landmarks_return_safe_result():
87     safe = [False] * 5
88     assert fingers_state(None) == safe
89     assert fingers_state([]) == safe
90     assert fingers_state([(0, 0)]) * (EXPECTED_LANDMARKS - 1) == safe
91     assert fingers_state([None] * EXPECTED_LANDMARKS) == safe
92     assert fingers_state(OPEN_HAND)[0] is True # unchanged by the guard tests
93
94
95 # -----
96 # Palm geometry
97 # -----

```

Four malformed inputs — None, an empty list, a list one element too short, and a list of 21 None values — all asserted to return exactly `[False] * 5`. The final line re-tests a valid hand, guarding against the possibility that a guard was written so aggressively that it suppresses correct input too. That is the shape of a complete boundary test: every invalid case *and* one valid case in the same test, so a regression in either direction fails.

5.3.4 The fake MediaPipe results

```

298 class FakeLandmark:
299     def __init__(self, x, y, z=0.0):
300         self.x, self.y, self.z = x, y, z
301
302
303 class FakeHand:
304     def __init__(self, landmarks):
305         self.landmark = landmarks
306
307
308 class FakeClassification:

```

```

309     def __init__(self, label, score=0.9):
310         self.label, self.score = label, score
311
312
313     class FakeClassificationList:
314         def __init__(self, label):
315             self.classification = [FakeClassification(label)]
316
317
318     class FakeResults:
319         def __init__(self, hands=(), handedness=None, include_handedness=True):
320             if hands is not None:
321                 self.multi_hand_landmarks = list(hands)
322             if include_handedness:
323                 self.multi_handedness = list(handedness or [])
324
325
326     WIDTH = HEIGHT = 1000
327
328
329     def _fake_hand(pixel_points):
330         return FakeHand([FakeLandmark(x / WIDTH, y / HEIGHT, 0.0) for x, y in pixel_points])

```

These five tiny classes are the reason `parse\_hand\_data` is testable without MediaPipe installed. Each mimics exactly one level of the real result object's attribute chain: `FakeResults` holds the two lists, `FakeHand` wraps a landmark list, `FakeClassificationList` wraps a classification, and `FakeClassification` carries the label. The full dotted chain — from `multi\_handedness` through `classification[0]` down to `label` — therefore resolves exactly as it does in production, and the duck-typed parser cannot tell the difference.

`FakeResults` has one particularly useful parameter:

- `include\_handedness=False` omits the attribute *entirely* on construction, which is how the “missing `multi\_handedness`” branch is covered. Passing an empty list would test something different: an attribute that exists but is empty. Both are real scenarios, and both are tested.

5.3.5 Parse tests

```

333     def test_parse_two_hands():
334         results = FakeResults(
335             hands=[_fake_hand(OPEN_HAND), _fake_hand(FIST)],
336             handedness=[FakeClassificationList("Right"), FakeClassificationList("Left")],
337         )
338         hands = parse_hand_data(results, WIDTH, HEIGHT)
339
340         assert len(hands) == 2
341         assert [hand["label"] for hand in hands] == ["Right", "Left"]
342         assert [len(hand["px"]) for hand in hands] == [EXPECTED_LANDMARKS] * 2
343         assert len(hands[0]["norm"][0]) == 3
344
345         assert hands[0]["palm_z"] < 0
346         assert hands[0]["orientation"] == "Palm"
347
348         for hand, source in zip(hands, (OPEN_HAND, FIST)):
349             for got, want in zip(hand["px"], source):
350                 assert abs(got[0] - want[0]) <= 1 and abs(got[1] - want[1]) <= 1
351             xmin, ymin, xmax, ymax = hand["bbox"]
352             assert xmin == min(p[0] for p in source) or abs(xmin - min(p[0] for p in source)) <= 1
353             assert ymax <= max(p[1] for p in source) + 1

```

```

354
355
356 def test_parse_zero_hands():
357     assert parse_hand_data(FakeResults(hands=[], handedness=[]), WIDTH, HEIGHT) == []
358
359
360 def test_parse_none_and_empty_objects():
361     assert parse_hand_data(None, WIDTH, HEIGHT) == []
362     assert parse_hand_data(object(), WIDTH, HEIGHT) == []
363     assert parse_hand_data(FakeResults(hands=None, include_handedness=False), WIDTH, HEIGHT) ==
    []

```

- **test_parse_two_hands** asserts on the label list, the landmark counts, the three-element norm tuples, the sign of palm_z, and the orientation string — then compares every pixel coordinate against the source with a ± 1 tolerance. The tolerance is not sloppiness: the fake landmarks are built by dividing by 1000 and the parser multiplies by 1000 and truncates, so a half-unit difference is a floating-point artefact of the round trip, not a logic error. A genuinely wrong scale factor would be off by far more than one pixel.
- **test_parse_zero_hands** covers the empty-but-present lists.
- **test_parse_none_and_empty_objects** covers three distinct degenerate inputs: None, a bare object() with no attributes at all, and a result with neither list. All three must return [] rather than raising — the “never raises” contract is what allows the render loop to be written without a defensive try around it.

```

387 def test_parse_skips_incomplete_hand():
388     results = FakeResults(
389         hands=[_fake_hand(OPEN_HAND[:10]), _fake_hand(FIST)],
390         handedness=[FakeClassificationList("Right"), FakeClassificationList("Left")],
391     )
392     hands = parse_hand_data(results, WIDTH, HEIGHT)
393     assert len(hands) == 1
394     assert hands[0]["label"] == "Left"
395
396
397 def test_parse_returns_mirrored_and_back_facing_cases():
398     palm = parse_hand_data(FakeResults(hands=[_fake_hand(OPEN_HAND)]), WIDTH, HEIGHT)[0]
399     back = parse_hand_data(FakeResults(hands=[_fake_hand(_mirror_x(OPEN_HAND))]), WIDTH, HEIGHT)
400     assert (palm["orientation"], back["orientation"]) == ("Palm", "Back")

```

- **test_parse_skips_incomplete_hand** builds a two-hand result where the first hand has only ten landmarks, and asserts that exactly one hand survives — with the *second* hand’s label, proving the skip did not shift the handedness correspondence.
- **test_parse_returns_mirrored_and_back_facing_cases** is the end-to-end confirmation of Section 3.3: the same hand, mirrored in *x*, must flip the orientation label. It ties the geometry, the parser and the sign convention together in one assertion.

Chapter summary. The pure module converts coordinates once, applies three geometric rules and two temporal filters, and never touches hardware or raises on bad input. The application layer owns the model, the camera fallback chain, the HUD and the CLI, and routes both its live and headless paths through the same annotation and drawing functions. The tests encode the two hardest-won lessons of the project — rotation invariance for the

thumb, and a total, exception-free contract for malformed landmarks — as executable assertions rather than comments.

Complete Source Listings

The three files that make up the project, printed in full. These are the exact files from the repository — included here with `\textbackslash lstinputlisting`, not retyped — so the line numbers match those used throughout Chapter 5.

A.1 `hand\_logic.py`

The pure-logic module: no OpenCV, no MediaPipe, no camera. Everything in Chapter 3 lives here.

```

1  """Pure hand geometry, landmark parsing and gesture classification.
2
3  This module deliberately imports nothing but the standard library (`math`,
4  `collections`). That keeps every rule below unit-testable with no camera, no
5  OpenCV and no MediaPipe, and lets CI run the logic suite in seconds.
6
7  All pixel coordinates are `(x, y)` with the origin at the top-left of the
8  image, so `y` grows downward: a smaller `y` means "higher on screen".
9  """
10
11  from __future__ import annotations
12
13  import math
14  from collections import deque
15
16  WRIST = 0
17  INDEX_MCP = 5
18  PINKY_MCP = 17
19  THUMB_IP = 3
20  THUMB_TIP = 4
21  INDEX_TIP = 8
22
23  FINGER_PAIRS = [(8, 6), (12, 10), (16, 14), (20, 18)]
24
25  EXPECTED_LANDMARKS = 21
26
27  THUMB_DISTANCE = "distance"
28  THUMB_X = "x"
29
30  NO_GESTURE = ""
31
32
33  def _distance(a, b):
34      return math.hypot(a[0] - b[0], a[1] - b[1])
35
36
37  def to_pixel_landmarks(landmarks, width, height):
38      """Convert normalized landmarks to integer pixel coordinates."""
39      return [(int(lm.x * width), int(lm.y * height)) for lm in landmarks]
40
41
42  def bbox_from_pixels(pixel_landmarks):
43      """Bounding box `(xmin, ymin, xmax, ymax)` of a pixel landmark list."""
44      xs = [point[0] for point in pixel_landmarks]

```

```

45     ys = [point[1] for point in pixel_landmarks]
46     return (min(xs), min(ys), max(xs), max(ys))
47
48
49 def palm_normal_z(pixel_landmarks):
50     """Z component of the palm-plane normal, from wrist/index-MCP/pinky-MCP.
51
52     ``Z = (x5 - x0)(y17 - y0) - (y5 - y0)(x17 - x0)``
53
54     The sign says which side of the hand faces the camera, which is a useful
55     orientation diagnostic. The convention assumes a mirrored (selfie) view:
56     ``Z < 0`` means the palm faces the camera, ``Z > 0`` the back of the hand.
57     """
58     x0, y0 = pixel_landmarks[WRIST][0], pixel_landmarks[WRIST][1]
59     x5, y5 = pixel_landmarks[INDEX_MCP][0], pixel_landmarks[INDEX_MCP][1]
60     x17, y17 = pixel_landmarks[PINKY_MCP][0], pixel_landmarks[PINKY_MCP][1]
61     return float((x5 - x0) * (y17 - y0) - (y5 - y0) * (x17 - x0))
62
63
64 def orientation_from_palm_z(palm_z):
65     """Map the palm-normal sign to the HUD label ``"Palm"`` or ``"Back"``. """
66     return "Palm" if palm_z < 0 else "Back"
67
68
69 def _handedness_label(handedness, swap_handedness):
70     try:
71         label = handedness.classification[0].label
72     except (AttributeError, IndexError, TypeError):
73         return "Unknown"
74     if label not in ("Left", "Right"):
75         return "Unknown"
76     if swap_handedness:
77         return "Left" if label == "Right" else "Right"
78     return label
79
80
81 def parse_hand_data(results, width, height, swap_handedness=False):
82     """Extract everything we need from a MediaPipe result object, once.
83
84     Duck-typed on purpose: it only ever touches ``multi_hand_landmarks`` and
85     ``multi_handedness``, so a fake result object can drive it in tests.
86
87     Returns one dict per detected hand::
88
89         {"label": "Left"|"Right"|"Unknown", "norm": [(x, y, z), ...],
90          "px": [(x, y), ...], "bbox": (xmin, ymin, xmax, ymax),
91          "palm_z": float, "orientation": "Palm"|"Back"}
92
93     ``None``, empty and missing fields all yield ``[]`` this never raises.
94     Hands without a full 21-landmark set are skipped, since no rule below can
95     be evaluated for them.
96     """
97     hand_landmarks = getattr(results, "multi_hand_landmarks", None)
98     if not hand_landmarks:
99         return []
100
101     handedness = getattr(results, "multi_handedness", None) or []
102
103     hands = []
104     for index, hand in enumerate(hand_landmarks):
105         landmarks = getattr(hand, "landmark", None)
106         if not landmarks or len(landmarks) < EXPECTED_LANDMARKS:
107             continue

```

```

108     norm = [(lm.x, lm.y, lm.z) for lm in landmarks]
109     px = to_pixel_landmarks(landmarks, width, height)
110     z = palm_normal_z(px)
111     label = _handedness_label(handedness[index], swap_handedness) if index < len(handedness)
112     else "Unknown"
113
114     hands.append(
115         {
116             "label": label,
117             "norm": norm,
118             "px": px,
119             "bbox": bbox_from_pixels(px),
120             "palm_z": z,
121             "orientation": orientation_from_palm_z(z),
122         }
123     )
124     return hands
125
126
127 def fingers_state(pixel_landmarks, hand_label="Unknown", palm_z=None, thumb_mode=THUMB_DISTANCE):
128     """Return ``[thumb, index, middle, ring, pinky]`` extended flags.
129
130     Indexpinky use the vertical rule from the spec: the fingertip is extended
131     when it sits strictly above its PIP joint (``y_tip < y_pip``).
132
133     The thumb default is the distance rule, which is robust to handedness and to
134     rotation in the image plane: the tip is extended when it moves *away* from
135     the pinky knuckle relative to the IP joint. ``thumb_mode="x"`` keeps the
136     spec's original horizontal rule (``Right`` -> ``x_tip < x_ip``, ``Left`` ->
137     ``x_tip > x_ip``) available for reference; it is only valid for an upright,
138     palm-facing hand.
139
140     ``palm_z`` is accepted for API symmetry with the orientation readout; the
141     default distance rule does not need it (it is already handedness-agnostic).
142
143     A ``None``, malformed or short landmark list returns exactly ``[False] * 5``
144     and never raises.
145     """
146     if not pixel_landmarks or len(pixel_landmarks) < EXPECTED_LANDMARKS:
147         return [False] * 5
148
149     try:
150         states = [pixel_landmarks[tip][1] < pixel_landmarks[pip][1] for tip, pip in FINGER_PAIRS]
151
152         if thumb_mode == THUMB_X and hand_label in ("Left", "Right"):
153             tip, ip = pixel_landmarks[THUMB_TIP], pixel_landmarks[THUMB_IP]
154             thumb = tip[0] < ip[0] if hand_label == "Right" else tip[0] > ip[0]
155         else:
156             thumb = _distance(pixel_landmarks[THUMB_TIP], pixel_landmarks[PINKY_MCP]) > _distance
157             (
158                 pixel_landmarks[THUMB_IP], pixel_landmarks[PINKY_MCP]
159             )
160     except (IndexError, TypeError, ValueError):
161         return [False] * 5
162
163     return [thumb] + states
164
165 def classify_gesture(states):
166     """Name the pose from a 5-element finger-state list."""
167     if not states or len(states) < 5:
168         return NO_GESTURE

```

```

169     thumb, index, middle, ring, pinky = (bool(state) for state in states[:5])
170
171
172     if thumb and index and middle and ring and pinky:
173         return "Open Palm"
174     if not (thumb or index or middle or ring or pinky):
175         return "Fist"
176     if thumb and not (index or middle or ring or pinky):
177         return "Thumbs Up"
178     if not thumb and index and middle and not (ring or pinky):
179         return "Peace"
180     if thumb and index and not (middle or ring) and pinky:
181         return "Rock On"
182     return NO_GESTURE
183
184
185 class GestureDebouncer:
186     """Rolling majority vote that suppresses single-frame gesture flicker.
187
188     ``update`` returns the new label once at least ``min_agree`` of the last
189     ``window`` frames agree on it, and otherwise keeps returning the last stable
190     label. Before any majority has ever formed it returns ``""``.
191     """
192
193     def __init__(self, window=5, min_agree=4):
194         self.window = window
195         self.min_agree = min_agree
196         self._history = deque(maxlen=window)
197         self._stable = NO_GESTURE
198
199     @property
200     def stable(self):
201         return self._stable
202
203     def update(self, gesture):
204         self._history.append(gesture)
205         if self._history.count(gesture) >= self.min_agree:
206             self._stable = gesture
207         return self._stable
208
209
210 def ema(previous, current, alpha=0.65):
211     """Exponential moving average: ``alpha * current + (1 - alpha) * previous``.
212
213     ``previous is None`` (first sighting) returns ``current`` unchanged.
214     """
215     if previous is None:
216         return (current[0], current[1])
217     return (
218         alpha * current[0] + (1 - alpha) * previous[0],
219         alpha * current[1] + (1 - alpha) * previous[1],
220     )

```

A.2 hand_tracker.py

The application: the model, the capture loop, the HUD, the CLI, the camera fallback chain, and the headless -image mode.

```

1  """Real-time hand and finger tracker.
2
3  Captures a webcam feed, extracts 21 3D hand landmarks per hand with MediaPipe,
4  decides which fingers are extended, and overlays the skeleton, a smoothed index
5  fingertip marker and a telemetry HUD on the video.
6
7  ``-image PATH`` runs the exact same pipeline over a single still instead of a
8  camera, which makes the whole thing verifiable (and CI-testable) without a
9  webcam.
10
11  Run ``python hand_tracker.py -help`` for the full flag list.
12  """
13
14  from __future__ import annotations
15
16  import argparse
17  import logging
18  import os
19  import time
20  import warnings
21  from collections import deque
22  from datetime import datetime
23
24  import cv2
25  import numpy as np
26
27  from hand_logic import (
28      INDEX_TIP,
29      GestureDebouncer,
30      classify_gesture,
31      ema,
32      fingers_state,
33      parse_hand_data,
34  )
35
36  LOG = logging.getLogger("hand_tracker")
37
38  with warnings.catch_warnings():
39      warnings.simplefilter("ignore")
40      import mediapipe as mp
41
42  if not hasattr(mp, "solutions") or not hasattr(mp.solutions, "hands"):
43      raise RuntimeError(
44          "mediapipe>=0.10.30 removed mp.solutions, which this app is built on. "
45          "Install the pinned version instead: pip install -r requirements.txt "
46          "(do NOT `pip install -U mediapipe`)."
47      )
48
49  with warnings.catch_warnings():
50      warnings.simplefilter("ignore")
51      mp_hands = mp.solutions.hands
52      mp_drawing = mp.solutions.drawing_utils
53
54  WINDOW_NAME = "Hand Tracker"
55  CAPTURES_DIR = "captures"
56
57  DEFAULT_WIDTH = 640
58  DEFAULT_HEIGHT = 480

```

```

59 HUD_FONT = cv2.FONT_HERSHEY_SIMPLEX
60 HUD_FONT_SCALE = 0.6
61 HUD_FONT_THICKNESS = 1
62 HUD_MARGIN = 10
63 HUD_PAD = 10
64 HUD_ALPHA = 0.6
65
66
67 INDEX_TIP_HALO_RADIUS = 14
68 INDEX_TIP_RADIUS = 10
69 INDEX_TIP_COLOR = (0, 165, 255)
70
71 FPS_WINDOW = 30
72
73
74 class HandTracker:
75     """MediaPipe Hands wrapper with a single-pass, per-frame data cache."""
76
77     def __init__(
78         self,
79         max_hands=2,
80         mode=False,
81         det_conf=0.7,
82         track_conf=0.6,
83         thumb_mode="distance",
84         swap_handedness=False,
85     ):
86         self.thumb_mode = thumb_mode
87         self.swap_handedness = swap_handedness
88         self.hands_data = []
89         self._smooth_tip = {}
90         self._debounce = {}
91         self.hands = mp_hands.Hands(
92             static_image_mode=mode,
93             max_num_hands=max_hands,
94             min_detection_confidence=det_conf,
95             min_tracking_confidence=track_conf,
96         )
97
98     def find_hands(self, frame, draw=True):
99         """Detect hands in a BGR frame, draw the mesh, and rebuild the cache.
100
101         The normalized landmarks, pixel landmarks, bounding box, handedness
102         label and palm orientation are computed once here and reused by every
103         helper below for the rest of the frame.
104         """
105         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
106         rgb.flags.writeable = False
107         results = self.hands.process(rgb)
108         rgb.flags.writeable = True
109
110         height, width = frame.shape[:2]
111         self.hands_data = parse_hand_data(results, width, height, self.swap_handedness)
112
113         if draw and results.multi_hand_landmarks:
114             for hand_landmarks in results.multi_hand_landmarks:
115                 mp_drawing.draw_landmarks(
116                     frame,
117                     hand_landmarks,
118                     mp_hands.HAND_CONNECTIONS,
119                     mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2, circle_radius=2),
120                     mp_drawing.DrawingSpec(color=(235, 235, 235), thickness=2),
121                 )

```

```

122     return frame
123
124     def hand_count(self):
125         """Number of hands seen in the most recent ``find_hands`` call."""
126         return len(self.hands_data)
127
128     def hand_label(self, hand_index):
129         """Handedness label of a detected hand, or ``"Unknown"``."""
130         if 0 <= hand_index < len(self.hands_data):
131             return self.hands_data[hand_index]["label"]
132         return "Unknown"
133
134     def find_positions(self, frame, hand_index=0):
135         """Cached pixel landmarks for one hand, or ``[]`` if there is none.
136
137         ``frame`` is only kept for signature compatibility: the returned points
138         were computed inside the last ``find_hands`` call, so they are only
139         valid for a frame of that same size.
140         """
141         if 0 <= hand_index < len(self.hands_data):
142             return list(self.hands_data[hand_index]["px"])
143         return []
144
145     def fingers_up(self, landmarks, hand_label):
146         """``[thumb, index, middle, ring, pinky]`` extended flags for one hand."""
147         return fingers_state(landmarks, hand_label, thumb_mode=self.thumb_mode)
148
149     def index_tip(self, hand_index, alpha=0.65):
150         """EMA-smoothed pixel position of landmark 8, or ``None``."""
151         if not 0 <= hand_index < len(self.hands_data):
152             return None
153
154         hand = self.hands_data[hand_index]
155         if len(hand["px"]) <= INDEX_TIP:
156             return None
157
158         key = self._state_key(hand_index, hand["label"])
159         smoothed = ema(self._smooth_tip.get(key), hand["px"][INDEX_TIP], alpha)
160         self._smooth_tip[key] = smoothed
161         return (int(smoothed[0]), int(smoothed[1]))
162
163     def orientation(self, hand_index):
164         """``"Palm"`` or ``"Back"`` for a detected hand, or ``""``."""
165         if 0 <= hand_index < len(self.hands_data):
166             return self.hands_data[hand_index]["orientation"]
167         return ""
168
169     def gesture(self, hand_index, states):
170         """Debounce gesture name for the finger states of one hand."""
171         key = self._state_key(hand_index, self.hand_label(hand_index))
172         debouncer = self._debounce.get(key)
173         if debouncer is None:
174             debouncer = GestureDebounce()
175             self._debounce[key] = debouncer
176         return debouncer.update(classify_gesture(states))
177
178     @staticmethod
179     def _state_key(hand_index, label):
180         """Key per-hand smoothing/debounce state.
181
182         MediaPipe's hand list order is not stable across frames, so keying by
183         list index would blend two different physical hands together when they
184         reorder the label is a much steadier identity.

```

```

185         """
186         return label if label in ("Left", "Right") else f"index:{hand_index}"
187
188     def close(self):
189         self.hands.close()
190
191
192 def hud_lines(total_fingers, hand_lines, fps_text):
193     """Assemble the HUD text: FPS, total count, then one line per hand."""
194     return [f"FPS: {fps_text}", f"Fingers: {total_fingers}"] + list(hand_lines)
195
196
197 def draw_hud(frame, lines):
198     """Draw an alpha-blended card behind white telemetry text (top-left)."""
199     if not lines:
200         return frame
201
202     sizes = [cv2.getTextSize(line, HUD_FONT, HUD_FONT_SCALE, HUD_FONT_THICKNESS)[0] for line in
203               lines]
204     text_width = max(width for width, _ in sizes)
205     text_height = max(height for _, height in sizes)
206     line_height = text_height + 12
207
208     height, width = frame.shape[:2]
209     x0, y0 = HUD_MARGIN, HUD_MARGIN
210     x1 = min(width, x0 + text_width + 2 * HUD_PAD)
211     y1 = min(height, y0 + 2 * HUD_PAD + line_height * len(lines))
212     if x1 <= x0 or y1 <= y0:
213         return frame
214
215     card = frame[y0:y1, x0:x1]
216     frame[y0:y1, x0:x1] = cv2.addWeighted(card, 1.0 - HUD_ALPHA, np.zeros_like(card), HUD_ALPHA,
217                                           0)
218
219     y = y0 + HUD_PAD + text_height
220     for line in lines:
221         cv2.putText(
222             frame,
223             line,
224             (x0 + HUD_PAD, y),
225             HUD_FONT,
226             HUD_FONT_SCALE,
227             (255, 255, 255),
228             HUD_FONT_THICKNESS,
229             cv2.LINE_AA,
230             )
231         y += line_height
232     return frame
233
234 def annotate_hands(tracker, frame):
235     """Run the finger logic and overlays for every detected hand.
236
237     Returns the total number of extended fingers and one HUD line per hand.
238     """
239     total = 0
240     lines = []
241
242     for index in range(tracker.hand_count()):
243         label = tracker.hand_label(index)
244         landmarks = tracker.find_positions(frame, index)
245         states = tracker.fingers_up(landmarks, label)
246         count = sum(states)

```

```

246         total += count
247         lines.append(f"[label]: {count} ({tracker.gesture(index, states)}) [{tracker.orientation(
index)}]")
248
249         tip = tracker.index_tip(index)
250         if tip is not None:
251             cv2.circle(frame, tip, INDEX_TIP_HALO_RADIUS, (255, 255, 255), cv2.FILLED)
252             cv2.circle(frame, tip, INDEX_TIP_RADIUS, INDEX_TIP_COLOR, cv2.FILLED)
253
254     return total, lines
255
256
257 def save_screenshot(frame):
258     """Write the annotated frame (HUD burned in) to ``captures/``."""
259     os.makedirs(CAPTURES_DIR, exist_ok=True)
260     stamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")[:-3]
261     path = os.path.join(CAPTURES_DIR, f"frame_{stamp}.png")
262     if cv2.imwrite(path, frame):
263         LOG.info("Saved screenshot: %s", path)
264     else:
265         LOG.error("Could not write screenshot: %s", path)
266
267
268 def open_camera(index, width, height):
269     """Open a webcam, falling back across backends and camera indices."""
270     backends = [cv2.CAP_DSHOW, cv2.CAP_MSMF, cv2.CAP_ANY] if os.name == "nt" else [cv2.CAP_ANY]
271
272     for candidate in [index] + [other for other in (1, 0) if other != index]:
273         for backend in backends:
274             capture = cv2.VideoCapture(candidate, backend)
275             if capture.isOpened():
276                 capture.set(cv2.CAP_PROP_FRAME_WIDTH, width)
277                 capture.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
278                 LOG.info(
279                     "Camera %d opened at %dx%d.",
280                     candidate,
281                     int(capture.get(cv2.CAP_PROP_FRAME_WIDTH)),
282                     int(capture.get(cv2.CAP_PROP_FRAME_HEIGHT)),
283                 )
284                 return capture
285             capture.release()
286             LOG.warning("Camera %d did not open.", candidate)
287
288     LOG.error("Could not open a webcam. Check it is connected and not in use by another app.")
289     return None
290
291
292 def run_live(args):
293     """Webcam capture loop: read, flip, analyse, draw, show."""
294     capture = open_camera(args.camera, args.width or DEFAULT_WIDTH, args.height or DEFAULT_HEIGHT)
295     if capture is None:
296         return 1
297
298     tracker = HandTracker(
299         max_hands=args.max_hands,
300         mode=args.static,
301         thumb_mode=args.thumb_mode,
302         swap_handedness=args.swap_handedness,
303     )
304     frame_times = deque(maxlen=FPS_WINDOW)
305     previous_time = time.time()
306

```

```

307     try:
308         while True:
309             ok, frame = capture.read()
310             if not ok:
311                 LOG.error("Frame capture failed; stopping.")
312                 break
313
314             frame = cv2.flip(frame, 1)
315             frame = tracker.find_hands(frame, draw=not args.no_draw)
316             total, hand_lines = annotate_hands(tracker, frame)
317
318             now = time.time()
319             delta = now - previous_time
320             previous_time = now
321             if delta > 0:
322                 frame_times.append(1.0 / delta)
323             fps_text = f"{sum(frame_times) / len(frame_times):.0f}" if frame_times else "0"
324
325             draw_hud(frame, hud_lines(total, hand_lines, fps_text))
326             cv2.imshow(WINDOW_NAME, frame)
327
328             key = cv2.waitKey(1) & 0xFF
329             if key in (ord("q"), 27):
330                 break
331             if key == ord("s"):
332                 save_screenshot(frame)
333         finally:
334             capture.release()
335             tracker.close()
336             cv2.destroyAllWindows()
337
338     return 0
339
340
341 def run_image(args):
342     """Run the full pipeline once over a still image and write ``<stem>_out.png``."""
343     frame = cv2.imread(args.image)
344     if frame is None:
345         LOG.error("Could not read image: %s", args.image)
346         return 1
347
348     if args.width and args.height:
349         frame = cv2.resize(frame, (args.width, args.height))
350         LOG.info("Resized input to %dx%d.", args.width, args.height)
351
352     tracker = HandTracker(
353         max_hands=args.max_hands,
354         mode=True,
355         thumb_mode=args.thumb_mode,
356         swap_handedness=args.swap_handedness,
357     )
358     try:
359         frame = tracker.find_hands(frame, draw=not args.no_draw)
360         total, hand_lines = annotate_hands(tracker, frame)
361         draw_hud(frame, hud_lines(total, hand_lines, "n/a (still)"))
362         hands_seen = tracker.hand_count()
363     finally:
364         tracker.close()
365
366     stem, _ = os.path.splitext(args.image)
367     output_path = f"{stem}_out.png"
368     if not cv2.imwrite(output_path, frame):
369         LOG.error("Could not write output image: %s", output_path)

```

```

370         return 1
371
372     LOG.info("Detected %d hand(s) with %d extended finger(s); wrote %s", hands_seen, total,
373             output_path)
374     return 0
375
376 def build_parser():
377     parser = argparse.ArgumentParser(
378         prog="hand_tracker.py",
379         description="Real-time hand and finger tracker (MediaPipe + OpenCV).",
380     )
381     parser.add_argument("--camera", type=int, default=0, help="webcam index (default: 0)")
382     parser.add_argument(
383         "--width", type=int, default=None, help="capture width (default: 640 live, native in -
384         image)"
385     )
386     parser.add_argument(
387         "--height", type=int, default=None, help="capture height (default: 480 live, native in -
388         image)"
389     )
390     parser.add_argument("--max-hands", type=int, default=2, help="maximum hands to detect (default
391     : 2)")
392     parser.add_argument("--no-draw", action="store_true", help="disable the skeletal mesh overlay")
393     parser.add_argument(
394         "--thumb-mode",
395         choices=("distance", "x"),
396         default="distance",
397         help="thumb rule: 'distance' (rotation-robust, default) or 'x' (original horizontal rule)
398     ",
399     )
400     parser.add_argument(
401         "--swap-handedness",
402         action="store_true",
403         help="swap Left/Right labels (display-only fix if your mirrored feed reports them
404         inverted)",
405     )
406     parser.add_argument("--static", action="store_true", help="use MediaPipe static_image_mode (no
407     tracking)")
408     parser.add_argument("--image", metavar="PATH", help="process a still image instead of the
409     camera, then exit")
410     parser.add_argument(
411         "--log-level",
412         default="INFO",
413         choices=("DEBUG", "INFO", "WARNING", "ERROR"),
414         help="logging verbosity (default: INFO)",
415     )
416     return parser
417
418 def main(argv=None):
419     args = build_parser().parse_args(argv)
420     logging.basicConfig(level=getattr(logging, args.log_level), format="%(levelname)s: %(message)
421     s")
422
423     if args.image:
424         return run_image(args)
425     return run_live(args)
426
427 if __name__ == "__main__":
428     raise SystemExit(main())

```

A.3 tests/test_hand_logic.py

The offline suite. Synthetic coordinates only — it runs in a bare Python interpreter with no camera and no third-party packages installed.

```

1  """Offline tests for the pure logic in ``hand_logic.py``.
2
3  No camera, no OpenCV, no MediaPipe: the synthetic hands below are plain
4  coordinate lists, so this file runs in a bare Python environment.
5  """
6
7  import math
8
9  from hand_logic import (
10     EXPECTED_LANDMARKS,
11     GestureDebouncer,
12     NO_GESTURE,
13     THUMB_X,
14     bbox_from_pixels,
15     classify_gesture,
16     ema,
17     fingers_state,
18     orientation_from_palm_z,
19     palm_normal_z,
20     parse_hand_data,
21     to_pixel_landmarks,
22 )
23
24 # -----
25 # Synthetic hands. Coordinates are pixels in a mirrored (selfie) view with the
26 # origin at the top-left, so y grows downward. Index 0 = wrist.
27 # -----
28
29 #: Right hand, palm facing the camera, fingers up, thumb extended sideways.
30 OPEN_HAND = [
31     (320, 420), # 0  wrist
32     (365, 400), # 1  thumb CMC
33     (395, 370), # 2  thumb MCP
34     (415, 345), # 3  thumb IP
35     (430, 320), # 4  thumb TIP (far from the pinky knuckle -> extended)
36     (355, 320), # 5  index MCP
37     (350, 280), # 6  index PIP
38     (348, 260), # 7  index DIP
39     (346, 235), # 8  index TIP (above its PIP -> extended)
40     (320, 315), # 9  middle MCP
41     (318, 270), # 10 middle PIP
42     (317, 245), # 11 middle DIP
43     (316, 215), # 12 middle TIP
44     (290, 320), # 13 ring MCP
45     (285, 280), # 14 ring PIP
46     (283, 258), # 15 ring DIP
47     (281, 230), # 16 ring TIP
48     (262, 335), # 17 pinky MCP
49     (252, 305), # 18 pinky PIP
50     (247, 285), # 19 pinky DIP
51     (243, 262), # 20 pinky TIP
52 ]
53
54 #: Closed fist: every tip curled below its PIP, thumb folded across the palm.
55 FIST = [
56     (320, 420), # 0  wrist
57     (355, 395), # 1  thumb CMC
58     (375, 375), # 2  thumb MCP

```

```

59     (395, 355), # 3 thumb IP
60     (375, 350), # 4 thumb TIP (closer to the pinky knuckle -> folded)
61     (355, 320), # 5 index MCP
62     (350, 280), # 6 index PIP
63     (350, 297), # 7
64     (352, 305), # 8 index TIP
65     (320, 315), # 9 middle MCP
66     (318, 270), # 10 middle PIP
67     (319, 288), # 11
68     (320, 296), # 12 middle TIP
69     (290, 320), # 13 ring MCP
70     (285, 280), # 14 ring PIP
71     (286, 298), # 15
72     (287, 306), # 16 ring TIP
73     (262, 335), # 17 pinky MCP
74     (252, 305), # 18 pinky PIP
75     (253, 322), # 19
76     (254, 330), # 20 pinky TIP
77 ]
78
79
80 def _with_finger(points, tip_index, tip_position):
81     updated = list(points)
82     updated[tip_index] = tip_position
83     return updated
84
85
86 def _with_thumb(points, cmc, mcp, ip, tip):
87     updated = list(points)
88     updated[1], updated[2], updated[3], updated[4] = cmc, mcp, ip, tip
89     return updated
90
91
92 def _rotate(points, degrees):
93     """Rotate every landmark around the set's centroid."""
94     radians = math.radians(degrees)
95     cos_a, sin_a = math.cos(radians), math.sin(radians)
96     cx = sum(p[0] for p in points) / len(points)
97     cy = sum(p[1] for p in points) / len(points)
98     return [
99         (
100             round(cx + cos_a * (x - cx) - sin_a * (y - cy)),
101             round(cy + sin_a * (x - cx) + cos_a * (y - cy)),
102         )
103         for x, y in points
104     ]
105
106
107 def _mirror_x(points):
108     return [(-x, y) for x, y in points]
109
110
111 # -----
112 # Finger state
113 # -----
114
115
116 def test_all_fingers_extended():
117     assert fingers_state(OPEN_HAND) == [True, True, True, True, True]
118
119
120 def test_all_fingers_folded():
121     assert fingers_state(FIST) == [False, False, False, False, False]

```

```

122
123
124 def test_index_only_extended():
125     hand = _with_finger(FIST, 8, (346, 235))
126     assert fingers_state(hand) == [False, True, False, False, False]
127
128
129 def test_each_finger_in_isolation():
130     for tip, position, expected_index in (
131         (12, (316, 215), 2),
132         (16, (281, 230), 3),
133         (20, (243, 262), 4),
134     ):
135         states = fingers_state(_with_finger(FIST, tip, position))
136         expected = [False] * 5
137         expected[expected_index] = True
138         assert states == expected, f"finger {tip} not detected in isolation"
139
140
141 def test_thumb_only_extended():
142     hand = _with_thumb(FIST, (365, 400), (395, 370), (415, 345), (430, 320))
143     assert fingers_state(hand) == [True, False, False, False, False]
144
145
146 def test_thumb_distance_mode_extended_and_folded():
147     assert fingers_state(OPEN_HAND)[0] is True
148     assert fingers_state(FIST)[0] is False
149
150
151 def test_thumb_x_mode_right_hand():
152     hand = _with_thumb(FIST, (0, 0), (0, 0), (200, 100), (100, 100))
153     assert fingers_state(hand, "Right", thumb_mode=THUMB_X)[0] is True
154
155
156 def test_thumb_x_mode_left_hand():
157     hand = _with_thumb(FIST, (0, 0), (0, 0), (200, 100), (300, 100))
158     assert fingers_state(hand, "Left", thumb_mode=THUMB_X)[0] is True
159
160
161 def test_thumb_x_mode_wrong_side_is_folded():
162     hand = _with_thumb(FIST, (0, 0), (0, 0), (200, 100), (300, 100))
163     assert fingers_state(hand, "Right", thumb_mode=THUMB_X)[0] is False
164
165
166 def test_thumb_x_mode_unknown_label_falls_back_to_distance():
167     # OPEN_HAND's thumb sits at x4=430 > x3=415, so the horizontal rule folds it
168     # for a "Right" label while the distance rule extends it.
169     assert fingers_state(OPEN_HAND, "Right", thumb_mode=THUMB_X)[0] is False
170     assert fingers_state(OPEN_HAND, "Unknown", thumb_mode=THUMB_X)[0] is True
171
172
173 def test_thumb_survives_image_plane_rotation():
174     """The regression test for the old x-only rule: rotate the hand 90 degrees."""
175     for degrees in (90, 37, 180, -120):
176         rotated = _rotate(OPEN_HAND, degrees)
177         assert fingers_state(rotated)[0] is True, f"thumb lost at {degrees} degrees"
178
179
180 def test_thumb_state_is_handedness_agnostic():
181     for label in ("Left", "Right", "Unknown"):
182         assert fingers_state(OPEN_HAND, label)[0] is True
183         assert fingers_state(FIST, label)[0] is False
184

```

```

185
186 def test_malformed_landmarks_return_safe_result():
187     safe = [False] * 5
188     assert fingers_state(None) == safe
189     assert fingers_state([]) == safe
190     assert fingers_state([(0, 0)] * (EXPECTED_LANDMARKS - 1)) == safe
191     assert fingers_state([None] * EXPECTED_LANDMARKS) == safe
192     assert fingers_state(OPEN_HAND)[0] is True # unchanged by the guard tests
193
194
195 # -----
196 # Palm geometry
197 # -----
198
199
200 def test_palm_normal_sign_flips_between_palm_and_back():
201     palm_facing = palm_normal_z(OPEN_HAND)
202     back_facing = palm_normal_z(_mirror_x(OPEN_HAND))
203     assert palm_facing < 0 < back_facing
204
205
206 def test_orientation_from_palm_z():
207     assert orientation_from_palm_z(palm_normal_z(OPEN_HAND)) == "Palm"
208     assert orientation_from_palm_z(palm_normal_z(_mirror_x(OPEN_HAND))) == "Back"
209     assert orientation_from_palm_z(0.0) == "Back"
210
211
212 def test_bbox_from_pixels():
213     assert bbox_from_pixels(OPEN_HAND) == (243, 215, 430, 420)
214
215
216 def test_to_pixel_landmarks_scales_and_truncates():
217     class Landmark:
218         def __init__(self, x, y, z):
219             self.x, self.y, self.z = x, y, z
220
221     landmarks = [Landmark(0.25, 0.5, 0.0), Landmark(0.999, 0.001, -0.2)]
222     assert to_pixel_landmarks(landmarks, 640, 480) == [(160, 240), (639, 0)]
223
224
225 # -----
226 # Gesture classification
227 # -----
228
229
230 def test_classify_gesture_known_poses():
231     assert classify_gesture([True] * 5) == "Open Palm"
232     assert classify_gesture([False] * 5) == "Fist"
233     assert classify_gesture([True, False, False, False, False]) == "Thumbs Up"
234     assert classify_gesture([False, True, True, False, False]) == "Peace"
235     assert classify_gesture([True, True, False, False, True]) == "Rock On"
236
237
238 def test_classify_gesture_unknown_and_malformed():
239     assert classify_gesture([False, False, True, True, True]) == NO_GESTURE
240     assert classify_gesture([]) == NO_GESTURE
241     assert classify_gesture(None) == NO_GESTURE
242
243
244 def test_classify_gesture_from_real_landmarks():
245     assert classify_gesture(fingers_state(OPEN_HAND)) == "Open Palm"
246     assert classify_gesture(fingers_state(FIST)) == "Fist"
247

```

```

248 # -----
249 # Debouncer and EMA
250 # -----
251
252
253
254 def test_debouncer_waits_for_min_agree():
255     debouncer = GestureDebouncer()
256     assert [debouncer.update("Peace") for _ in range(3)] == [NO_GESTURE] * 3
257     assert debouncer.update("Peace") == "Peace"
258
259
260 def test_debouncer_ignores_single_frame_blip():
261     debouncer = GestureDebouncer()
262     for _ in range(5):
263         debouncer.update("Fist")
264     assert debouncer.update("Open Palm") == "Fist"
265     assert debouncer.update("Fist") == "Fist"
266
267
268 def test_debouncer_switches_after_sustained_change():
269     debouncer = GestureDebouncer()
270     for _ in range(5):
271         debouncer.update("Peace")
272     for _ in range(3):
273         assert debouncer.update("Fist") == "Peace"
274     assert debouncer.update("Fist") == "Fist"
275
276
277 def test_debouncer_window_is_bounded():
278     debouncer = GestureDebouncer(window=3, min_agree=2)
279     assert debouncer.update("Peace") == NO_GESTURE
280     assert debouncer.update("Peace") == "Peace"
281     assert len(debouncer._history) <= 3
282
283
284 def test_ema_first_sample_passes_through():
285     assert ema(None, (10, 20)) == (10, 20)
286
287
288 def test_ema_weights_current_sample():
289     assert ema((0, 0), (100, 100), 0.65) == (65.0, 65.0)
290     assert ema((65, 65), (100, 100), 0.65) == (87.75, 87.75)
291
292
293 # -----
294 # Parsing (duck-typed fake MediaPipe results)
295 # -----
296
297
298 class FakeLandmark:
299     def __init__(self, x, y, z=0.0):
300         self.x, self.y, self.z = x, y, z
301
302
303 class FakeHand:
304     def __init__(self, landmarks):
305         self.landmark = landmarks
306
307
308 class FakeClassification:
309     def __init__(self, label, score=0.9):
310         self.label, self.score = label, score

```

```

311
312
313 class FakeClassificationList:
314     def __init__(self, label):
315         self.classification = [FakeClassification(label)]
316
317
318 class FakeResults:
319     def __init__(self, hands=(), handedness=None, include_handedness=True):
320         if hands is not None:
321             self.multi_hand_landmarks = list(hands)
322         if include_handedness:
323             self.multi_handedness = list(handedness or [])
324
325
326 WIDTH = HEIGHT = 1000
327
328
329 def _fake_hand(pixel_points):
330     return FakeHand([FakeLandmark(x / WIDTH, y / HEIGHT, 0.0) for x, y in pixel_points])
331
332
333 def test_parse_two_hands():
334     results = FakeResults(
335         hands=[_fake_hand(OPEN_HAND), _fake_hand(FIST)],
336         handedness=[FakeClassificationList("Right"), FakeClassificationList("Left")],
337     )
338     hands = parse_hand_data(results, WIDTH, HEIGHT)
339
340     assert len(hands) == 2
341     assert [hand["label"] for hand in hands] == ["Right", "Left"]
342     assert [len(hand["px"]) for hand in hands] == [EXPECTED_LANDMARKS] * 2
343     assert len(hands[0]["norm"][0]) == 3
344
345     assert hands[0]["palm_z"] < 0
346     assert hands[0]["orientation"] == "Palm"
347
348     for hand, source in zip(hands, (OPEN_HAND, FIST)):
349         for got, want in zip(hand["px"], source):
350             assert abs(got[0] - want[0]) <= 1 and abs(got[1] - want[1]) <= 1
351         xmin, ymin, xmax, ymax = hand["bbox"]
352         assert xmin == min(p[0] for p in source) or abs(xmin - min(p[0] for p in source)) <= 1
353         assert ymax <= max(p[1] for p in source) + 1
354
355
356 def test_parse_zero_hands():
357     assert parse_hand_data(FakeResults(hands=[], handedness=[]), WIDTH, HEIGHT) == []
358
359
360 def test_parse_none_and_empty_objects():
361     assert parse_hand_data(None, WIDTH, HEIGHT) == []
362     assert parse_hand_data(object(), WIDTH, HEIGHT) == []
363     assert parse_hand_data(FakeResults(hands=None, include_handedness=False), WIDTH, HEIGHT) == []
364
365
366 def test_parse_missing_handedness_is_unknown():
367     results = FakeResults(hands=[_fake_hand(OPEN_HAND)], include_handedness=False)
368     hands = parse_hand_data(results, WIDTH, HEIGHT)
369     assert len(hands) == 1
370     assert hands[0]["label"] == "Unknown"
371
372

```

```

373 def test_parse_short_handedness_list_is_unknown():
374     results = FakeResults(hands=[_fake_hand(OPEN_HAND), _fake_hand(FIST)], handedness=[])
375     assert [hand["label"] for hand in parse_hand_data(results, WIDTH, HEIGHT)] == ["Unknown", "
    Unknown"]
376
377
378 def test_parse_swap_handedness():
379     results = FakeResults(
380         hands=[_fake_hand(OPEN_HAND), _fake_hand(FIST)],
381         handedness=[FakeClassificationList("Right"), FakeClassificationList("Left")],
382     )
383     hands = parse_hand_data(results, WIDTH, HEIGHT, swap_handedness=True)
384     assert [hand["label"] for hand in hands] == ["Left", "Right"]
385
386
387 def test_parse_skips_incomplete_hand():
388     results = FakeResults(
389         hands=[_fake_hand(OPEN_HAND[:10]), _fake_hand(FIST)],
390         handedness=[FakeClassificationList("Right"), FakeClassificationList("Left")],
391     )
392     hands = parse_hand_data(results, WIDTH, HEIGHT)
393     assert len(hands) == 1
394     assert hands[0]["label"] == "Left"
395
396
397 def test_parse_returns_mirrored_and_back_facing_cases():
398     palm = parse_hand_data(FakeResults(hands=[_fake_hand(OPEN_HAND)]), WIDTH, HEIGHT)[0]
399     back = parse_hand_data(FakeResults(hands=[_fake_hand(_mirror_x(OPEN_HAND))]), WIDTH, HEIGHT)
    [0]
400     assert (palm["orientation"], back["orientation"]) == ("Palm", "Back")

```

Verification Checklist and Troubleshooting

B.1 Setup and first run

#	Step	What success looks like
1	<code>python -m venv .venv</code>	A <code>.venv</code> directory appears.
2	activate, then <code>pip install -r requirements.txt</code>	Three packages install: <code>mediapipe 0.10.14</code> , <code>opencv-contrib-python 4.11.0.86</code> , <code>numpy 1.26.x</code> .
3	<code>pytest -q</code>	All tests pass. No camera is touched.
4	<code>python hand_tracker.py -image tests/fixtures/hand.png</code>	Prints a hand count and writes <code>tests/fixtures/hand_out.png</code> ; exit code 0.
5	<code>python hand_tracker.py</code>	A window titled <i>Hand Tracker</i> opens with the mirrored feed.

B.2 Live-mode acceptance tests

Test	Pre	Expected
Single hand	Raise one hand fingers up.	HUD shows one line; the count matches your fingers.
Two hands	Raise both hands	Two lines, and the total is the sum.
Wrist rotation	Rotate your wrists and turn the back of your hand toward the camera.	The thumb count must not flip. This is the fix the distance rule exists for.
Marker stability	Hold still and watch the index finger dot	It sits steady; no visible vibration.
Gesture debounce	Make a Peace sign then a Fist	The label changes once, after a few frames — no flicker through intermediate states.
Empty scene	Remove your hand cover the lens	Count goes to 0; FPS keeps updating; no crash.
Screenshot	Press s.	A timestamped PNG appears in captures/ with the HUD burned in.
Clean exit	Press q.	Window closes, camera light goes off, prompt returns.
Fallback	python hand-camera	Logs a warning per failed index, then either opens another camera or exits with a clear message

B.3 Troubleshooting

Symptom	Cause and fix
Could not open a webcam	Another application holds the device (video call, browser tab, or a previous run that was killed). Close it and retry, or use <code>-camera 1</code> .
<code>RuntimeError: mediapipe>=0.10.30 removed mp.solutions</code>	MediaPipe was upgraded. Reinstall with <code>pip install -r requirements.txt</code> .
<code>ImportError: libGL.so.1 (Linux)</code>	Missing system libraries for the non-headless OpenCV wheel: <code>sudo apt-get install -y libgl1 libglu1-mesa-dev</code> .
Handedness labels look inverted	Expected on a mirrored feed. Display-only: relaunch with <code>-swap-handedness</code> .
Thumb never detects with <code>-thumb-mode x</code>	That rule assumes an upright, palm-facing hand. Use the default distance mode.
The dot is drawn in the wrong place	The frame changed size between <code>find_hands</code> and <code>find_positions</code> ; the cache is valid for one frame size only (Section 4.2).
Low FPS	Reduce <code>-width/-height</code> , or pass <code>-no-draw</code> to skip the mesh.

Glossary and Quick Reference

C.1 Glossary

Alpha blending Combining two images by a weighted average, $(1 - \alpha)I_1 + \alpha I_2$. Used for the translucent HUD card with $\alpha = 0.6$.

Anchor A predefined box in a single-shot detector. BlazePalm refines anchors into palm boxes.

BGR The channel order OpenCV uses: blue, green, red. MediaPipe expects RGB, hence the one explicit conversion per frame.

BlazePalm The stage-1 detector. Finds palm boxes, not fingers — palms are rigid, so they are easier to detect robustly.

Cross product A vector operation producing a vector perpendicular to two inputs. Its z -component here gives the signed area of the palm triangle, whose sign reveals which side of the hand faces the camera.

Debouncing Requiring several consecutive agreeing observations before accepting a state change. Used on gestures: 4 of the last 5 frames.

Duck typing Accepting any object that has the attributes you use, regardless of its class. This is what makes `parse\_hand\_data` testable with a fake result object.

EMA Exponential moving average: a weighted blend of the current measurement and the previous smoothed value, with geometrically decaying memory. Used on the index fingertip with $\alpha = 0.65$.

Euclidean distance Straight-line distance, $d = \sqrt{\Delta x^2 + \Delta y^2}$. The basis of the thumb rule.

Frame One still image from a video stream.

FPS Frames per second — the loop's throughput. Averaged over a 30-frame window.

HUD Head-up display: the text overlay showing FPS, finger count, gesture and orientation.

Inference Running a trained model on new input. Training is not part of this project.

Landmark One of the 21 named 3D points the model predicts per hand.

MCP / PIP / DIP / TIP Knuckle, first finger joint, second finger joint, fingertip.

Normalized coordinates Positions in $[0, 1]$ relative to the image size, so the model is resolution-independent.

Palm normal The vector perpendicular to the palm plane; its z -component's sign says whether the palm or the back faces the camera.

ROI Region of interest — the rectangular slice of an image being processed, here the HUD card area.

Single-pass caching Computing derived telemetry once per frame and reusing it, rather than recomputing per call.

Static image mode MediaPipe configuration that treats each frame as an unrelated still. Required for `-image` mode.

Tensor A multi-dimensional array of numbers. A frame is a 3D tensor of shape $(H, W, 3)$.

C.2 Constants and defaults

Name	Value	Where	Meaning
<code>min_detection_confidence</code>	0.7	tracker	Confidence to accept a <i>new</i> hand.
<code>min_tracking_confidence</code>	0.6	tracker	Confidence to keep <i>following</i> a hand.
<code>max_num_hands</code>	2	tracker	Hands searched for simultaneously.
<code>EXPECTED_LANDMARKS</code>	21	logic	Validity gate for a landmark list.
<code>FINGER_PAIRS</code>	$(8, 6), (12, 10), (16, 14), (20, 18)$	logic	Tip and PIP per long finger.
<code>THUMB_TIP / THUMB_IP / PINKY_MCP</code>	4 / 3 / 17	logic	The thumb rule's three points.
<code>INDEX_TIP</code>	8	logic	The landmark carrying the smoothed marker.
<code>alpha (EMA)</code>	0.65	logic	Smoothing weight on the newest sample.
<code>window / min_agree</code>	5 / 4	logic	Debouncer window and required agreement.
<code>HUD_ALPHA</code>	0.6	tracker	Overlay weight in the HUD blend.
<code>FPS_WINDOW</code>	30	tracker	Frames averaged for the FPS readout.
<code>DEFAULT_WIDTH / HEIGHT</code>	640 / 480	tracker	Live capture size.

C.3 Dependency reference

Package	Pin	Why this exact version
mediapipe	0.10.14	Last line with the legacy <code>mp.solutions.hands</code> API; removed in 0.10.30+. Load-bearing — do not upgrade.
opencv-contrib-python	4.11.0.86	The superset build MediaPipe depends on; avoids two competing <code>cv2</code> installs.
numpy	<2	MediaPipe 0.10.x wheels are compiled against the NumPy 1.x ABI.
Python	3.9–3.12	3.13 has no wheel for MediaPipe 0.10.14. Tested on 3.12.

C.4 Files in the project

File	Purpose
<code>hand_logic.py</code>	Pure logic: parsing, finger and thumb rules, palm normal, EMA, gestures, debouncing. Standard library only.
<code>hand_tracker.py</code>	Application: HandTracker, capture loop, HUD, CLI, camera fallback, <code>-image</code> mode.
<code>tests/test_hand_logic.py</code>	Offline geometry and parse tests. No camera, no MediaPipe.
<code>tests/test_hand_tracker.py</code>	Cache and HUD tests; skips itself when <code>cv2/mediapipe</code> are absent.
<code>conftest.py</code>	Puts the project root on <code>sys.path</code> so <code>pytest</code> works as a bare console script.
<code>requirements.txt</code>	The three pins.
<code>run.bat</code>	Double-click launcher: uses the <code>venv</code> interpreter, passes arguments through.
<code>doc/build.bat</code>	Rebuilds this handbook (two <code>pdflatex</code> passes).
<code>.github/workflows/ci.yml</code>	Logic tests on 3.11/3.12, plus a full-dependency pipeline smoke test.